

долговременного хранения результатов и их использования при подготовке научных публикаций и отчётов. В отличие от разрозненных файловых хранилищ, единая база данных позволяет задать строгую структуру для описания запусков, зафиксировать связи между численными результатами, схемами синхронизации и использованными в работе теоретическими моделями. Это делает возможным не только оперативный поиск нужной информации, но и трассировку взаимосвязей между экспериментами, параметрами и текстовыми материалами, что особенно важно при многолетних проектах и коллективной работе нескольких исследователей.

С учётом многообразия задач, решаемых с помощью хаотических генераторов, информационная система должна поддерживать разные сценарии использования одной и той же базы данных. Для задач криптографии и генерации псевдослучайных последовательностей ключевыми становятся параметры, определяющие статистические свойства выходного сигнала и устойчивость схемы к внешним воздействиям; в этом случае требуется детальное хранение распределений, корреляционных функций и параметров тестов случайности. Для задач моделирования физических процессов важнее связь между параметрами модели, граничными условиями и экспериментальными данными, с которыми проводится сопоставление, поэтому особое внимание уделяется фиксации исходных предположений, единиц измерения и используемых приближений. Единая структура базы данных должна позволять хранить такие разнородные сведения в согласованном виде, не навязывая пользователю избыточных ограничений, но обеспечивая при этом достаточную формализацию для автоматизированной обработки и анализа.

Важным аспектом является также организация доступа к данным и разграничение прав пользователей. В типичной исследовательской группе присутствуют администраторы системы, разработчики математических моделей, инженеры-экспериментаторы и студенты, выполняющие учебные задания. Для каждой категории пользователей могут быть заданы собственные роли: одни имеют право изменять структуру справочников литературы и публиковать утверждённые отчёты, другие ограничены запуском моделирования и созданием черновых записей. Наличие ролевой модели доступа, реализованной на уровне информационной системы, позволяет обеспечить целостность базы данных, предотвратить случайное удаление важных записей и при необходимости организовать аудит действий пользователей. Это особенно актуально при подготовке официальных отчётов и статей, где необходимо гарантировать корректность приводимых результатов и ссылок на первичные данные.

2 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ СИСТЕМЫ

2.1 Проектирование базы данных

Проектирование базы данных приложения можно начать с *ER*-диаграммы в нотации Питера Чена. *ER*-диаграммы графически наглядно показывают, как взаимодействуют объекты в базе данных. Существует немало нотаций для построения таких диаграмм, однако наиболее часто используется нотация Питера Чена. Несмотря на трудоёмкость создания диаграммы в данной нотации, она хорошо отражает общую концепцию и идею базы данных и позволяет выделить имеющиеся сущности и их атрибуты. Для обозначения сущности используется прямоугольник, ее атрибутов – овалы, а отношений между сущностями – ромбы. Имена ключевых атрибутов подчеркиваются.

В базе данных можно выделить следующие сущности:

- пользователь;
- параметрический запуск (расчет траектории);
- символьное представление (алфавит);
- последовательность символов;
- статистика последовательностей;
- отчет о запуске;
- математическая формула / публикация (справочная информация).

Каждая из перечисленных сущностей описывается набором атрибутов, отражающих значимые для приложения характеристики. Например, для сущности «Пользователь» можно задать следующие атрибуты:

- логин;
- хеш пароля;
- электронная почта;
- имя;
- фамилия;
- роль (тип пользователя, определяющий права доступа).

Сущность «Параметрический запуск» описывает один эксперимент или моделирование системы Лоренца с заданными параметрами и начальными условиями. Для нее целесообразно выделить атрибуты:

- идентификатор запуска;
- дата и время выполнения;
- численные параметры модели (t_{max} , dt , $rratio$ и др.);
- начальные условия;
- имя режима (например, *App3* или *App4*);
- владелец (ссылка на пользователя).

Сущность «Алфавит» задает используемое символьное кодирование траектории. Ее атрибуты могут включать:

- идентификатор алфавита;
- набор символов;
- правила разбиения фазового пространства;
- текстовое описание.

Сущность «Последовательность символов» связана с конкретным запуском и алфавитом, поскольку символьное описание строится по результатам моделирования для выбранной схемы кодирования. Для данной сущности могут быть заданы:

- идентификатор последовательности;
- ссылка на параметрический запуск;
- ссылка на алфавит;
- длина последовательности;
- путь к сохраненному файлу либо само символьное представление.

Сущность «Статистика последовательностей» агрегирует вычисленные характеристики, полученные по одной или нескольким последовательностям. В качестве основных атрибутов можно использовать:

- идентификатор записи статистики;
- ссылка на последовательность;
- значения энтропий;
- показатели совпадения (*mean match*);
- параметры синхронизации (время синхронизации и др.);
- дополнительные числовые характеристики (распределения длин серий, матрицы переходов и т.п.).

Сущность «Отчет о запуске» отражает сформированные пользователем итоговые документы и файлы экспорта. Она может включать атрибуты:

- идентификатор отчета;
- ссылка на параметрический запуск;
- тип файла (*CSV, Excel, Word, PDF*);
- путь к файлу на диске;
- дата и время формирования;
- краткое текстовое описание.

Справочные сущности «Математическая формула» и «Публикация» используются для хранения теоретических сведений, связанных с анализом хаотических систем. Для них задаются:

- идентификатор записи;
- заголовок;
- текстовое или *LaTeX*-представление формулы;
- библиографическое описание или ссылка на источник;

– дополнительные примечания.

Связи между сущностями задаются таким образом, чтобы отразить реальные ограничения системы. Один пользователь может создавать множество запусков, но каждый запуск принадлежит ровно одному пользователю (связь типа « $I-M$ » между «Пользователь» и «Параметрический запуск»). Один запуск может иметь несколько связанных символьных последовательностей и вычисленных по ним статистик, однако каждая конкретная последовательность и каждая запись статистики относятся к одному запуску (отношение « $I-M$ » от «Параметрический запуск» к «Последовательность символов» и далее к «Статистика последовательностей»). Отчет также связан с единственным запуском, но для одного запуска может быть сформировано несколько отчетов разных типов.

Для более точного описания структуры данных в *ER*-диаграмму добавляются атрибуты-идентификаторы сущностей, которые обозначаются подчеркиванием, как ключевые атрибуты. Графическое представление диаграммы в нотации Питера Чена для базы данных системы анализа хаотических режимов Лоренца приводится на рисунке 2.1.

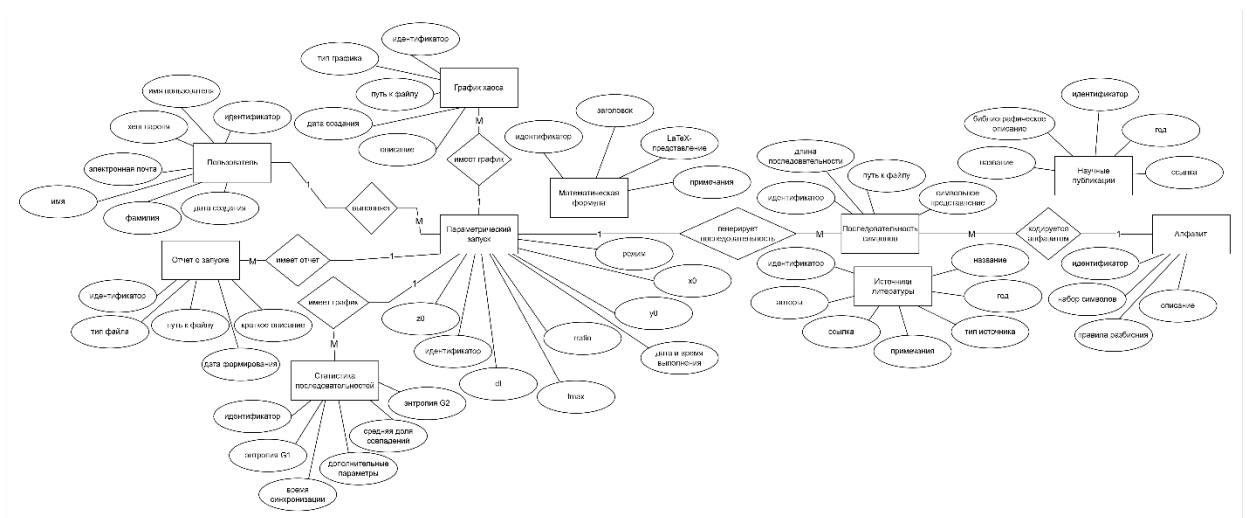


Рисунок 2.1 – Диаграмма в нотации Питера Чена

Следующим этапом проектирования базы данных является построение её логической схемы для системы анализа хаотических режимов Лоренца (см. рисунок 2.2). Логическая схема, основанная на *ER*-диаграмме в нотации Питера Чена, определяет набор таблиц и связей между ними и включает сущности, атрибуты, ключи и взаимосвязи, описывающие предметную область и бизнес-правила приложения [4]. Она не зависит от конкретной СУБД и используется для установления связей между объектами системы и способов работы с ними, а нормализация до третьей нормальной формы

обеспечивает атомарность хранения, снижение избыточности и устранение транзитивных зависимостей, что позволяет избежать аномалий при изменении данных.

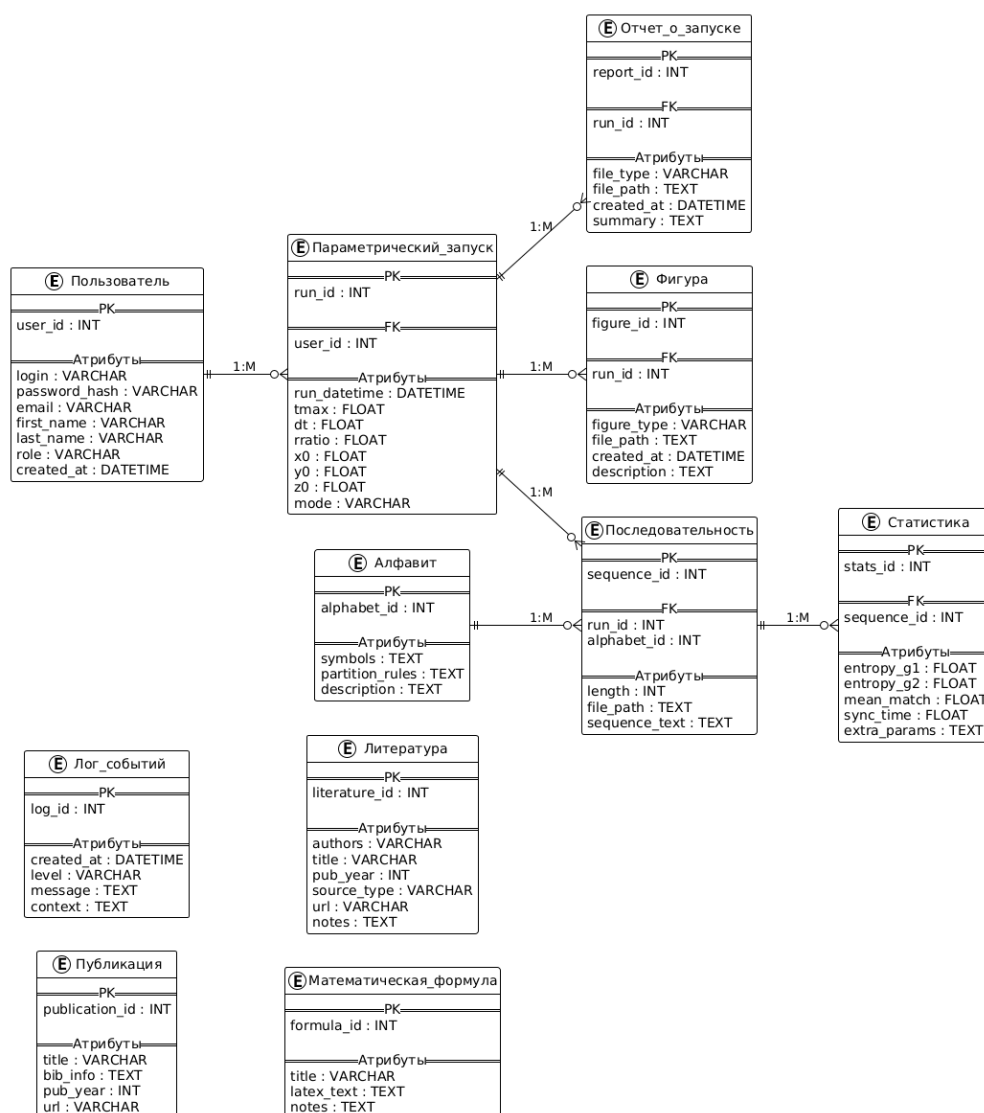


Рисунок 2.2 – Логическая схема базы данных

На схеме показаны таблицы будущей базы данных и связи между ними. Каждая таблица описывает свой тип объектов системы.

Есть таблица пользователей, где хранятся логин, пароль, имя, email и дата создания учётной записи. Пользователь связан с таблицей параметрических запусков так, что один пользователь может иметь много запусков, а каждый запуск принадлежит только одному пользователю. В таблице параметрических запусков хранятся параметры моделирования: моменты времени, коэффициенты, режим работы и дата запуска.

От каждого параметрического запуска идут связи к нескольким другим таблицам. С одним запуском могут быть связаны: несколько

последовательностей символов, несколько отчётов о запуске и несколько фигур (изображений результатов). В таблице отчётов лежат файлы отчётов и их краткое текстовое описание. В таблице фигур хранятся путь к файлу изображения, тип фигуры и описание.

Последовательность символов описывается отдельной таблицей: длина, путь к файлу и текст самой последовательности. Каждая последовательность связана с одной записью алфавита, но один алфавит может использоваться для многих последовательностей. В таблице алфавитов хранятся список символов, правила разбиения и текстовое описание.

Для каждой последовательности в таблице статистики может быть несколько записей статистических показателей. Там лежат, например, значения энтропии, меры совпадения и дополнительные параметры, относящиеся к конкретной обработке последовательности. Таким образом, одна последовательность имеет много связанных записей статистики.

Отдельно стоят служебные и справочные таблицы. Таблица логов событий хранит сообщения системы с временем, уровнем важности и текстом события. Таблица публикаций описывает публикации с заголовком, годом, журналом и примечаниями. Таблица литературы содержит источники: авторов, год, издательство и заметки. Таблица математических формул хранит формулы в виде текста и *LaTeX*-представления вместе с комментариями. Все эти таблицы могут использоваться для ссылок и оформления научной части, но напрямую не завязаны на основные сущности экспериментов.

2.2 Проектирование системы

Разрабатываемое веб-приложение для анализа хаотических режимов Лоренца строится на клиент-серверной архитектуре. Такая архитектура позволяет разделить систему на серверную часть, где выполняются основные вычисления, доступ к базе данных и генерация отчетов, и клиентскую часть, где пользователь взаимодействует с интерфейсом через веб-браузер. Клиент отправляет запросы на сервер (запуск численного эксперимента, просмотр результатов, формирование отчёта, работа с публикациями и формулами), сервер обрабатывает эти запросы, выполняет необходимые вычисления и операции с данными и возвращает результат клиенту. Основная вычислительная нагрузка при моделировании динамики, построении графиков и подготовке отчётов переносится с устройства пользователя на сервер.

В данной работе используется двухуровневая архитектура: сервер одновременно реализует функции веб-приложения и уровня доступа к базе данных. Такой подход оправдан, поскольку система ориентирована на

ограниченное число пользователей и не требует сложного горизонтального масштабирования. К преимуществам двухуровневой архитектуры относятся:

- возможность масштабировать систему за счёт увеличения ресурсов сервера;
- относительная экономия на оборудовании и инфраструктуре;
- снижение объёма сетевого трафика за счёт передачи клиенту только результатов расчётов и сформированных представлений данных, а не всего массива сырых данных.

К недостаткам двухуровневой архитектуры можно отнести:

- преимущественно вертикальное масштабирование, зависящее от производительности одного сервера;
- усложнение интеграции дополнительных сервисов и внешних систем по мере роста функциональности.

Проектирование системы начинается с построения *UML*-диаграмм, которые описывают архитектуру приложения, данные и взаимодействие объектов. Вначале разрабатывается диаграмма вариантов использования, отражающая бизнес-процессы и требования к системе анализа хаотической динамики. Эти диаграммы обеспечивают полное понимание проекта и служат отправной точкой для детальной проработки структуры, логики взаимодействий и дальнейшего проектирования модулей приложения, обеспечивая согласованность между этапами разработки и упрощая контроль выполнения технических требований системы. Тем самым формируется основа эффективного моделирования процессов и последующего анализа вычислительных характеристик системы.

Вариант использования описывает сценарий взаимодействия пользователя или внешней системы с приложением и перечень доступных функций (запуск моделирования, просмотр графиков, экспорт отчёта, работа с научными материалами) и ожидаемых результатов [5]. Таким образом, прецеденты фиксируют функциональные требования, а действующие лица представляют заинтересованные роли по отношению к системе.

Действующее лицо – это роль, которую объект внешней среды играет по отношению к системе. В проектируемом приложении действующим лицом может быть не только человек, но и, при необходимости, внешняя информационная система, инициирующая обмен данными или автоматический запуск расчётов по расписанию. На диаграмме варианты использования изображаются в виде эллипсов с подписями, а действующие лица – в виде стилизованных фигур человечков, соединённых с соответствующими прецедентами.

На рисунке 2.3 представлена диаграмма вариантов использования для разрабатываемого веб-приложения анализа хаотической динамики.

Диаграмма показывает, какие действия доступны различным категориям пользователей, какие результаты они получают и какие подсистемы при этом задействуются.

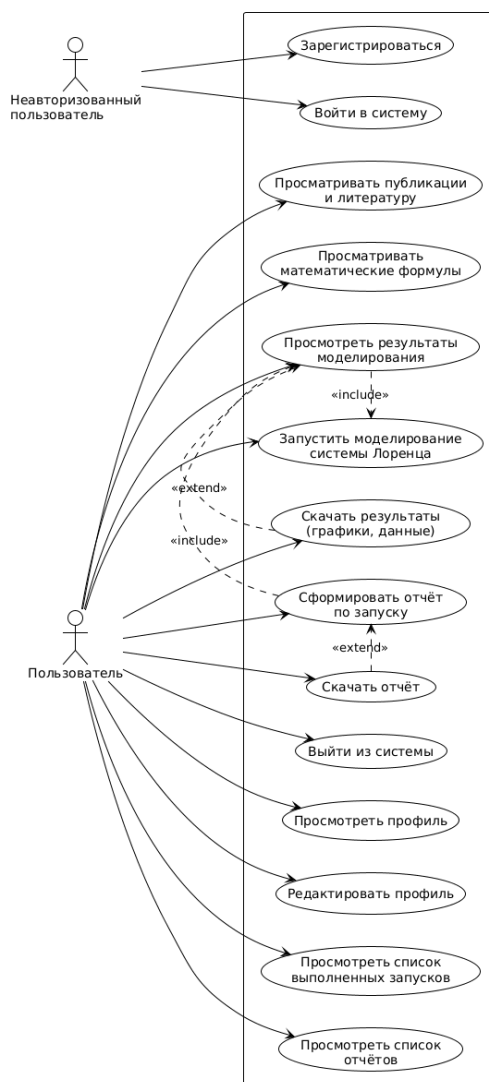


Рисунок 2.3 – Прецеденты пользователя

В разрабатываемой системе выделены следующие действующие лица:

- неавторизованный пользователь;
- авторизованный пользователь;
- администратор.

Исходя из потребностей этих ролей, для пользователя сформулированы следующие основные варианты использования:

- зарегистрироваться в системе;
- войти в профиль;
- выйти из профиля;
- просмотреть и изменить данные профиля;

- запустить моделирование системы Лоренца с заданными параметрами;
- просмотреть результаты моделирования (графики, последовательности, статистики);
- сохранить и скачать отчёт (*CSV, Excel, Word, PDF*);
- просмотреть список выполненных запусков и сформированных отчётов;
- просматривать справочную информацию: публикации, литературу, математические формулы.

Варианты использования, доступные администратору в системе анализа хаотических режимов, более обширны:

- войти в профиль;
- выйти из профиля;
- просмотреть данные профиля;
- изменить данные профиля;
- просмотреть список пользователей;
- добавить профиль администратора;
- удалить профиль пользователя;
- просмотреть список выполненных запусков системы Лоренца;
- просмотреть детали выбранного запуска (параметры, графики, статистики);
- удалить запуск из базы данных;
- просмотреть список сформированных отчётов;
- сформировать отчёт по выбранному запуску в требуемом формате (*CSV, Excel, Word, PDF*);
- удалить отчёт из базы данных;
- просмотреть и редактировать список публикаций;
- добавить публикацию;
- удалить публикацию;
- просмотреть и редактировать список литературных источников;
- добавить литературный источник;
- удалить литературный источник;
- просмотреть и редактировать список математических формул;
- добавить математическую формулу;
- удалить математическую формулу;
- управлять алфавитами и настройками кодирования последовательностей;
- импортировать данные в базу данных (например, запуски, отчёты, справочные материалы);

– экспортировать данные из базы данных для резервного копирования или последующей обработки.

Диаграмма вариантов использования для администратора представлена на рисунке 2.4.

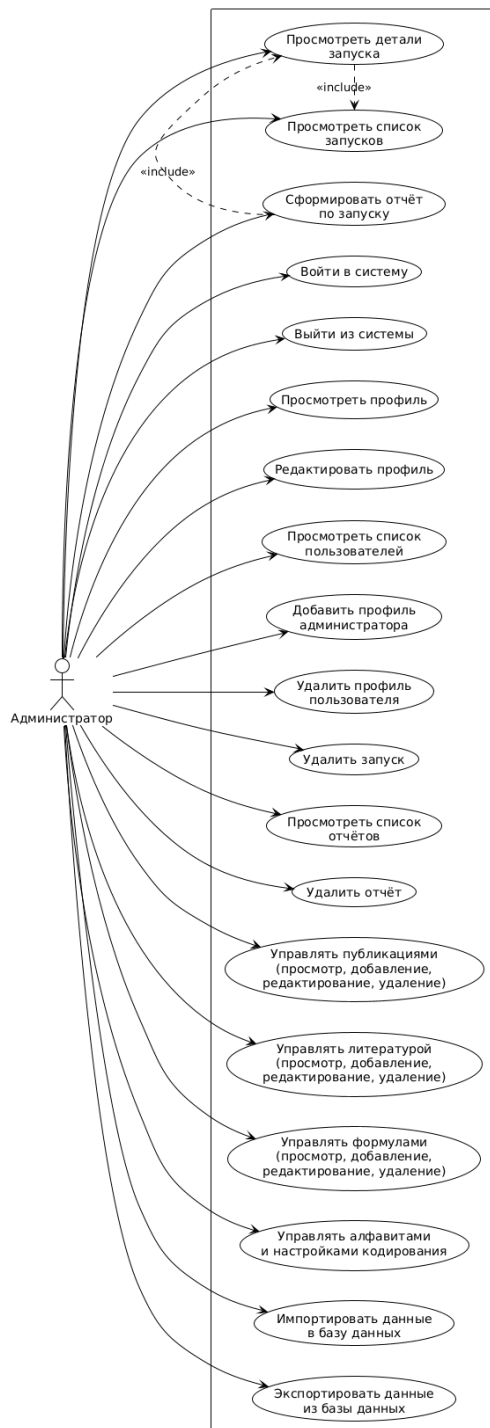


Рисунок 2.4 – Прецеденты администратора

Одной из ключевых *UML*-диаграмм для понимания архитектуры системы анализа хаотических режимов Лоренца является диаграмма

последовательности. Она показывает поток событий в рамках варианта использования и моделирует взаимодействие между объектами системы при выполнении таких функций, как запуск моделирования, сохранение результатов и формирование отчёта. Диаграмма наглядно отражает обмен сообщениями и их временной порядок, позволяя анализировать динамическое поведение системы и выявлять узкие места во взаимодействии компонентов.

Один вариант использования может иметь несколько последовательностей, поэтому обычно строится диаграмма для нормального, «успешного» сценария [6]. На ней показываются действующие лица и объекты, задействованные в сценарии (пользователь, веб-интерфейс, сервер приложения, модуль анализа Лоренца, база данных, модуль экспорта отчётов), а также сообщения между ними. Диаграмма последовательности для варианта использования «Запустить моделирование и сформировать отчёт» при успешном выполнении всех шагов показана на рисунке 2.5.

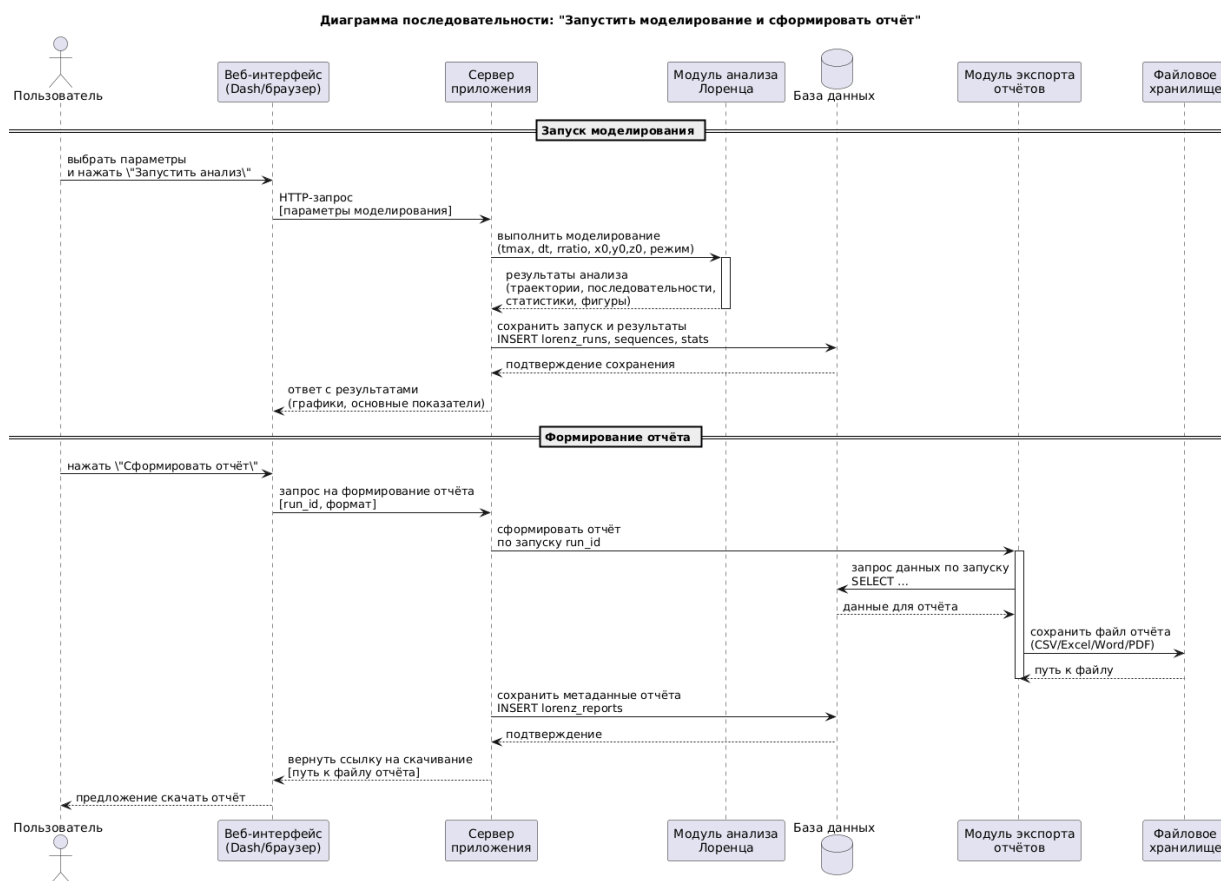


Рисунок 2.5 – Диаграмма последовательности для варианта использования «Запустить моделирование и сформировать отчёт»

Диаграмма классов также является одной из ключевых *UML*-диаграмм для понимания архитектуры разрабатываемой системы анализа хаотических режимов Лоренца. Она описывает статическую структуру приложения,

определяет типы классов, из которых состоит система, а также статические связи между ними. На диаграмме классов отображаются атрибуты и методы классов, а также ограничения, накладываемые на эти атрибуты и связи. Атрибут представляет собой именованное свойство класса, задающее диапазон возможных значений его экземпляров, а операция соответствует реализуемому методом поведению класса. Особое внимание при построении диаграммы классов уделяется типам отношений между классами.

Диаграммы классов используются не только для документирования и визуализации архитектуры, но и для конструирования системы посредством прямого и обратного проектирования, что делает их важным инструментом в процессе разработки. На рисунке 2.6 представлена диаграмма классов для разрабатываемой системы анализа хаотической динамики.

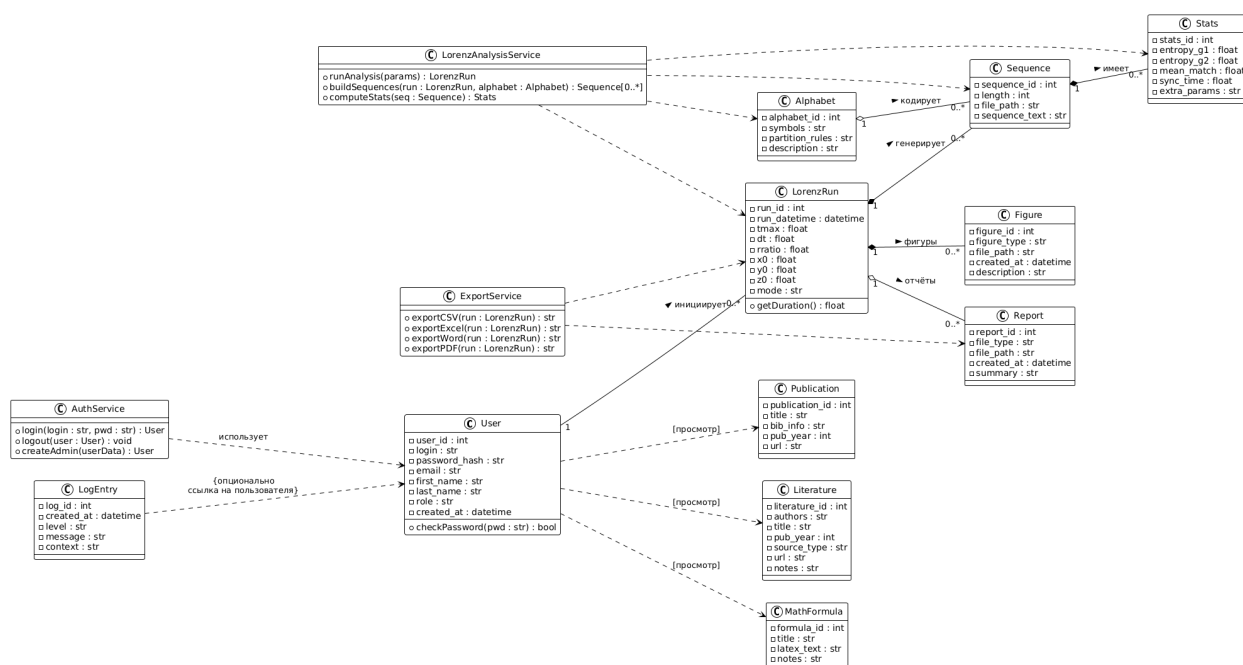


Рисунок 2.6 – Диаграмма классов

В системе выделяется набор основных классов (пользователь, параметрический запуск, последовательность символов, алфавит, статистика, отчёт, фигура, публикация, литературный источник, математическая формула), между которыми заданы отношения ассоциации, агрегации и композиции. Ассоциация описывает обычную семантическую связь между классами, например между пользователем и параметрическим запуском или между запуском и отчётом/фигурами, когда объекты связаны, но могут существовать независимо.

Для описания динамики работы системы вместо блок-схем используются *UML*-диаграммы состояний, отражающие изменения состояний

ключевых объектов и события, вызывающие переходы между ними, например для пользовательской сессии при регистрации нового пользователя (см. рисунок 2.7).

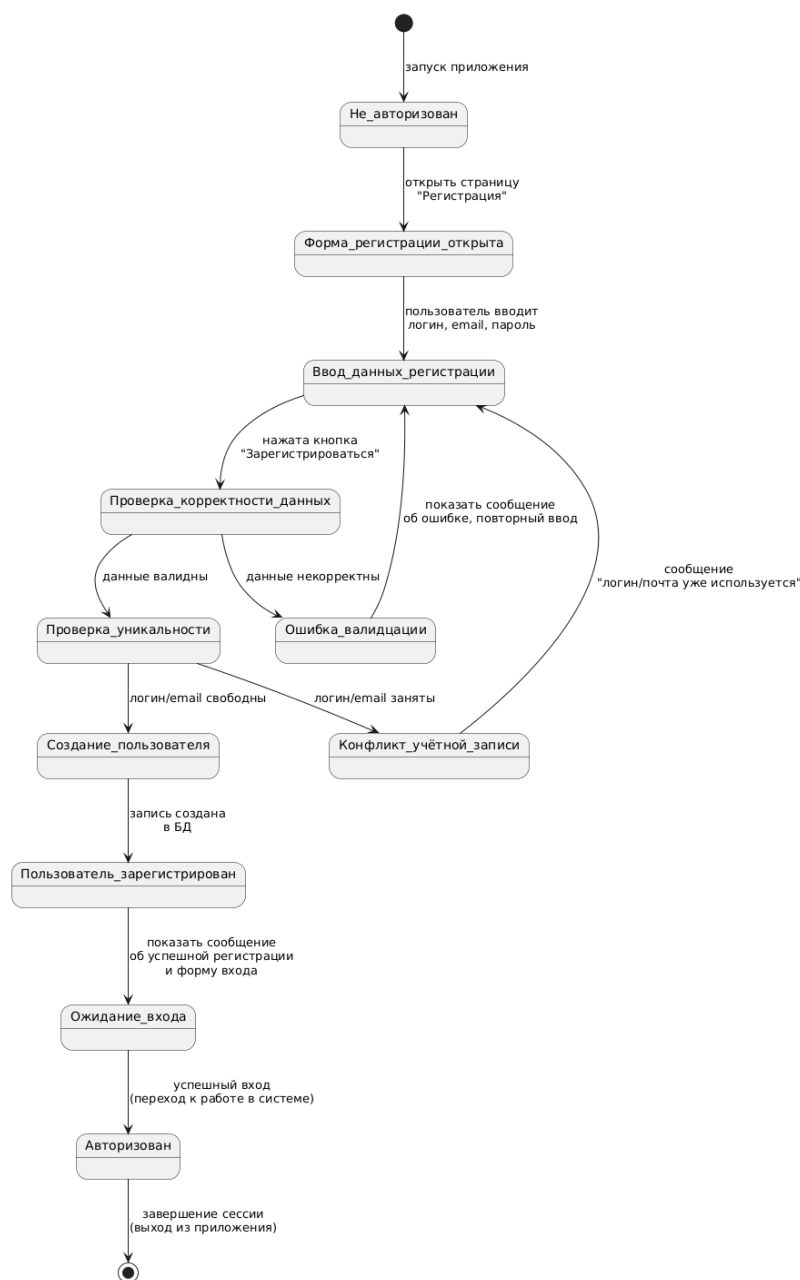


Рисунок 2.7 – Диаграмма состояний сценария регистрации пользователя

Жизненный цикл параметрического запуска системы Лоренца, от создания и задания параметров до моделирования, расчёта статистик и формирования отчёта, описывается диаграммой состояний, а не блок-схемой. На ней для объекта «Запуск» отражаются основные состояния (например, «создан», «подготовлен к расчёту», «выполняется», «завершён», «отчёт сформирован», «архивирован») и события, переводящие его между ними,

такие как запрос пользователя, успешное завершение расчёта, сохранение данных и команда формирования отчёта; диаграмма состояний для варианта использования «Запустить моделирование и сформировать отчёт» показана на рисунке 2.8.

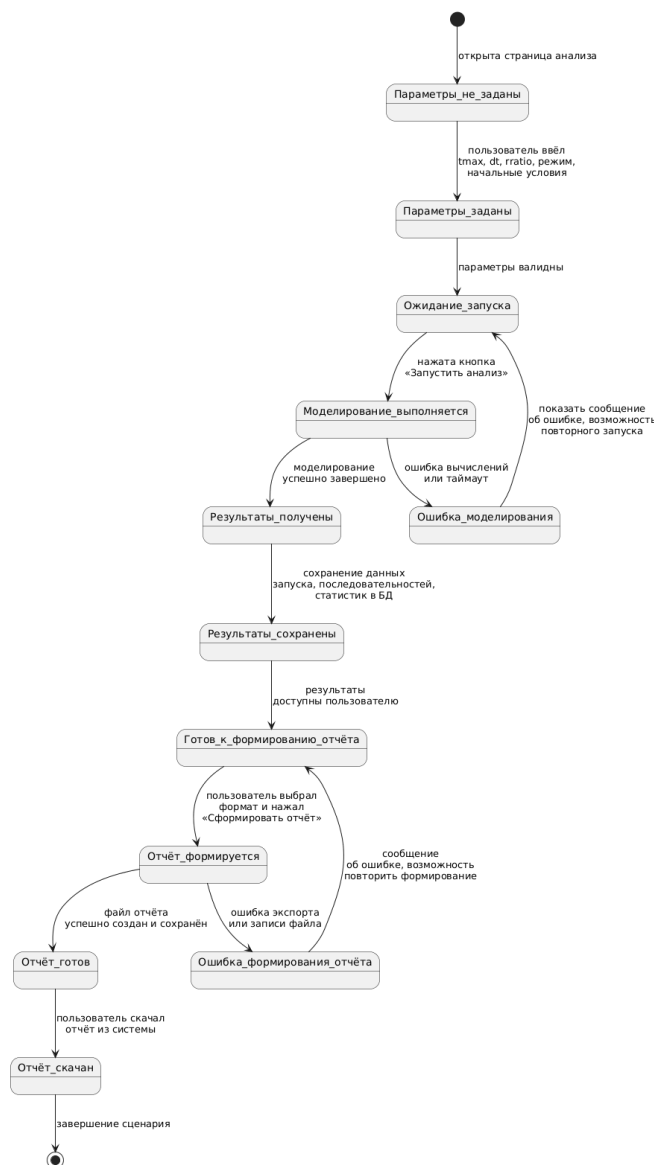


Рисунок 2.8 – Диаграмма состояний запуска моделирования

Диаграммы состояний выбраны вместо классических схем алгоритмов, так как позволяют сосредоточиться на изменении состояний объектов во времени, а не только на последовательности шагов. Они наглядно показывают реакции системы на разные события, ветвления и возвраты к предыдущим состояниям, что упрощает анализ долгих вычислительных процессов и позволяет лучше спроектировать обработку ошибок и восстановление работы приложения.

3 РАЗРАБОТКА БАЗЫ ДАННЫХ

3.1 Выбор средств реализации

При разработке информационной системы для исследования генератора Лоренца одним из наиболее ответственных этапов является выбор программно-технических средств. От того, насколько взвешенно будут приняты решения на этом этапе, зависит не только корректная работа приложения «здесь и сейчас», но и то, насколько легко систему можно будет поддерживать, адаптировать под новые требования и масштабировать при росте объёма данных и числа пользователей. Правильно подобранный стек позволяет безболезненно расширять функциональность, добавлять новые виды экспериментов, подключать внешние сервисы и при этом не переписывать уже реализованные модули. В данном разделе последовательно обосновывается выбор системы управления базами данных, языка серверной части и веб-фреймворка, а также показывается, какое место в общей архитектуре занимают технологии клиентского слоя.

В качестве основы подсистемы хранения данных рассматривались три варианта: реляционные СУБД *MySQL* и *PostgreSQL* и документно-ориентированная *MongoDB*. Реляционные СУБД традиционно демонстрируют высокую надёжность и удобны, когда информация представима в виде фиксированных таблиц с хорошо определёнными связями и ожидается преимущественно «классический» набор операций выборки и модификации [7]. Для системы генератора Лоренца структура данных иная: единичный запуск моделирования включает набор скалярных параметров (значения σ , ρ , β , шаг по времени, длительность эксперимента, начальные условия), массивы временных рядов координат $x(t)$, $y(t)$, $z(t)$, производные численные характеристики (показатели Ляпунова, статистики синхронизации, меры близости траекторий), а также набор ссылок на сформированные отчёты и изображения и связь с одной или несколькими научными публикациями. Если переносить всё это в реляционную модель, пришлось бы создавать отдельные таблицы для запусков, параметров, метрик, файлов, связей «запуск–публикация» и т.д., а для получения целостной картины по одному эксперименту постоянно выполнять несколько операций объединения. При росте числа экспериментов такая схема усложняется, становится менее наглядной и требует значительных усилий при любых изменениях модели данных.

Документно-ориентированная *MongoDB* гораздо лучше отражает динамичный и иерархический характер данных проекта. Каждый запуск моделирования может быть сохранён в виде одного документа коллекции

lorenz_runs, внутри которого компактно размещаются идентификатор запуска и ссылка на пользователя, набор параметров моделирования, краткое текстовое описание, вложенные структуры с рассчитанными метриками, списки путей к файлам изображений и отчётов, ссылки на связанные публикации и источники литературы. По аналогичному принципу организуются коллекции *lorenz_reports*, *publications*, *literature*, *figures*: каждый документ содержит базовый «каркас» полей (идентификатор, заголовок, тип объекта, временные метки) и произвольный набор дополнительных атрибутов, например теги, версию использованного алгоритма, статус публикации или перечень связанных запусков.

Гибкость схемы *MongoDB* особенно важна в контексте исследовательского проекта, где структура сохраняемых данных неизбежно развивается по мере появления новых идей и методов анализа. Если возникает необходимость сохранять, например, результаты нового алгоритма фазовой синхронизации или ещё один набор статистик, достаточно добавить в документ новый вложенный объект или массив, не изменяя уже существующие записи и не останавливая систему для миграции структуры. Это существенно снижает стоимость изменений по сравнению с реляционной моделью, где добавление новых сущностей и связей требует пересмотра нормализации и преобразования данных.

Отдельно следует выделить развитый механизм индексации в *MongoDB*. Для коллекции запусков можно создавать составные индексы по пользователю и дате запуска, по ключевым параметрам моделирования (например, по значению ρ , параметру связи, длительности эксперимента), по признаку «успешный/неуспешный» запуск или по идентификаторам связанных публикаций. Такие индексы позволяют быстро строить выборки для интерфейса (фильтрация по автору, интервалу времени, диапазонам параметров), формировать сводные отчёты по определённым сериям экспериментов, а также оперативно находить данные, использованные в конкретной статье или отчёте. Встроенная поддержка репликации, журналирования операций и механизмов резервного копирования даёт возможность настроить отказоустойчивое хранение и минимизировать риск утраты накопленных результатов многолетних исследований.

С учётом всех этих факторов – гибкой схемы, естественной поддержки вложенных структур и массивов, мощной системы индексов и встроенных средств обеспечения надёжности – в качестве основной СУБД для информационной системы исследования генератора Лоренца выбрана именно документно-ориентированная *MongoDB*. Такой выбор позволяет согласовать модель хранения данных с реальной структурой результатов экспериментов и обеспечивает запас прочности для дальнейшего развития проекта.

На этапе выбора языка программирования для серверной части сравнивались *Java*, *JavaScript (Node.js)* и *Python*. *Java* и *Node.js* обладают высокой производительностью, богатой экосистемой веб-фреймворков и хорошо подходят для построения высоконагруженных серверных приложений [8]. Однако в контексте данного проекта они имеют существенный недостаток: большинство численных алгоритмов и функций анализа хаотических систем уже реализованы на *Python*. Перенос этих алгоритмов на другой язык подвёл бы к дублированию кода, росту сложности тестирования и риску расхождения результатов между различными реализациями.

Python, напротив, изначально используется в проекте для интегрирования системы Лоренца, расчёта показателей Ляпунова, анализа синхронизации и построения визуализаций. Для него доступны специализированные библиотеки численного анализа и визуализации (*NumPy*, *SciPy*, *Pandas*, *Plotly*), что снимает необходимость реализации базовых алгоритмов «с нуля» и позволяет сосредоточиться на специфике предметной области. Использование одного языка как для научных расчётов, так и для серверной логики даёт возможность вынести существующие функции в отдельные модули и переиспользовать их в веб-приложении без переписывания, уменьшает объём кода и упрощает тестирование: результаты, полученные через интерфейс, совпадают с результатами автономных исследовательских скриптов.

Дальнейший выбор касался веб-фреймворка, который должен связать вычислительную часть и слой работы с базой данных в единое веб-приложение. Рассматривались три основных подхода: полнофункциональный фреймворк *Django*, высокопроизводительный асинхронный стек на базе *FastAPI* и минималистичная связка *Flask/Dash*. *Django* традиционно ориентирован на многостраничные сайты с реляционной БД и жёсткой моделью данных; его встроенная *ORM* и административная панель в значительной степени опираются на табличную структуру, что делает использование *Django* совместно с *MongoDB* менее естественным и приводит к дополнительным прослойкам между моделью данных и хранилищем. *FastAPI*, в свою очередь, отлично подходит для построения *REST*-ориентированных сервисов и микросервисной архитектуры, но требует разработки отдельного фронтенда на *JavaScript*-фреймворках (*React*, *Vue* и т.п.), что увеличивает стек используемых технологий и усложняет сопровождение учебного проекта [9].

С учётом этих ограничений была выбрана связка *Flask/Dash*, ориентированная на создание интерактивных аналитических приложений с минимальным количеством «ручного» фронтенда. *Flask* предоставляет лёгкий каркас для серверной части: маршрутизацию *HTTP*-запросов, подключение

механизмов аутентификации и авторизации, работу с сессиями пользователей, обработку форм и интеграцию с *MongoDB* через драйвер *PyMongo*. *Dash*, работающий поверх *Flask*, позволяет описывать пользовательский интерфейс декларативно в терминах компонентов (графики, таблицы, панели управления параметрами, элементы фильтрации) и автоматически связывать их с *Python*-функциями, реализующими бизнес-логику [10]. При изменении параметров моделирования, выборе другого запуска или публикации соответствующий *callback* вызывается на сервере, выполняет необходимые вычисления и запросы к базе данных и возвращает обновлённые данные для отображения в браузере. Такой подход избавляет от необходимости писать полноценный одностраничный фронтенд на *JavaScript* и позволяет сосредоточиться на корректности численных расчётов и структуре данных.

Клиентская часть приложения в этой архитектуре базируется на стандартных технологиях *HTML* и *CSS*, отвечающих за структуру страниц и оформление интерфейса. Компоненты *Dash* генерируют *HTML*-разметку и подключают необходимые стили, а за обработку событий (нажатие кнопок, изменение значений полей, выбор точек на графике) отвечает встроенный в браузер *JavaScript*, управляемый самим *Dash*. Разработчику при этом достаточно описывать поведение интерфейса через *Python-callback*-и: всё состояние приложения (выбранные параметры, результаты вычислений, кэшированные данные) находится на сервере, где доступны привычные средства логирования и отладки. При необходимости внешний вид можно детализировать с помощью собственных *CSS*-правил, а единичные элементы (например, модальные окна или всплывающие подсказки) – дополнить лёгкими скриптами, не усложняя общую архитектуру.

В совокупности выбранный стек – *MongoDB* для хранения данных, *Python* как основной язык реализации и связка *Flask/Dash* для веб-интерфейса формирует единое технологическое решение, хорошо согласованное с задачами численного моделирования и анализа хаотических систем. В рамках одной среды можно запускать эксперименты с генератором Лоренца, сохранять и структурировать их результаты, формировать отчёты и визуализации, управлять библиотекой публикаций и источников литературы, а также организовывать доступ пользователей с различными ролями. Однородность стека упрощает обучение и сопровождение: разработчик работает преимущественно с одним языком программирования и набором хорошо интегрированных библиотек, не переходя между несколькими несовместимыми технологиями.

Дополнительным преимуществом выбранного решения является использование свободно распространяемого и кроссплатформенного программного обеспечения. Система может быть развёрнута как на локальном

компьютере исследователя, так и на сервере лаборатории или в облачной инфраструктуре, при этом отсутствует необходимость в приобретении коммерческих лицензий. Активные сообщества вокруг *MongoDB*, *Python* и *Flask/Dash* обеспечивают наличие подробной документации, примеров и сторонних библиотек, что снижает риски при доработке приложения и облегчает интеграцию с внешними сервисами – системами контроля версий, хранилищами файлов, платформами совместной работы. В результате выбранные программно-технические средства оказываются не только достаточными для реализации текущих требований курсового проекта, но и создают задел для дальнейшего расширения системы исследования генератора Лоренца, добавления новых видов экспериментов и усложнения аналитических функций.

В результате выбранный стек – *MongoDB* для хранения данных, *Python* как основной язык реализации и связка *Flask/Dash* для веб-интерфейса – образует единое, хорошо согласованное решение, ориентированное на задачи численного моделирования и анализа хаотических систем. В рамках одной среды обеспечиваются запуск и управление экспериментами, долговременное хранение и структурирование результатов, формирование отчётов и визуализаций, ведение библиотеки публикаций и литературы, а также разграниченный доступ пользователей к данным. Использование свободно распространяемого и кроссплатформенного программного обеспечения позволяет развёртывать систему как локально, так и на сервере лаборатории или в облаке без приобретения коммерческих лицензий. Активные сообщества вокруг *MongoDB*, *Python* и *Flask/Dash* обеспечивают наличие обширной документации и примеров, что снижает риски при доработке и упрощает интеграцию с внешними сервисами. Всё это делает выбранные программно-технические средства не только достаточными для текущих задач курсового проекта, но и перспективной основой для дальнейшего развития системы исследования генератора Лоренца [11].

3.2 Реализация базы данных

Завершающим этапом проектирования является переход от логической модели к физической реализации базы данных в выбранной СУБД *MongoDB*. Физическая модель определяет конкретные коллекции, структуру документов, индексы и правила обеспечения целостности, с учётом особенностей документно-ориентированного хранилища и профиля запросов приложения. В отличие от реляционного подхода, где акцент делается на таблицах, столбцах и внешних ключах, здесь основными объектами являются коллекции документов и их вложенные поля; связи между сущностями чаще отражаются

через идентификаторы и денормализованные дубли полей, что позволяет уменьшить количество «соединений» на уровне приложения и ускорить типичные операции чтения.

В системе исследования генератора Лоренца выделены следующие ключевые коллекции: *users*, *lorenz_runs*, *lorenz_reports*, *publications*, *literature*, *figures*, *logs*, а также служебные коллекции для работы с сессиями и правами доступа. Каждая коллекция реализует одну или несколько логических сущностей предметной области и содержит документы со схожей структурой, но допускает наличие дополнительных полей при эволюции требований. Ниже описаны основные коллекции и типичные поля документов.

Коллекция *users* хранит информацию о пользователях системы (исследователях и администраторах). Типичный документ включает:

- *username* — уникальный логин пользователя;
- *password_hash* — хеш пароля с солью;
- *role* — роль пользователя (например, *admin*, *user*);
- *email* — адрес электронной почты;
- *created_at* — дата и время регистрации.

По полю *username* создаётся уникальный индекс, обеспечивающий однозначную идентификацию пользователя и ускоряющий операции аутентификации. Дополнительный индекс по полю *role* облегчает выборки в административных разделах (например, просмотр всех администраторов).

Коллекция *lorenz_runs* описывает запуски численного моделирования генератора Лоренца и является одной из центральных коллекций базы. Документ запуска содержит:

- *user* — ссылку (идентификатор или имя) на пользователя, инициировавшего запуск;
- *params* — вложенный объект с параметрами моделирования: t_{max} , dt , значения σ , ρ , β , начальные условия x_0, y_0, z_0 , параметры связи при синхронизации и др.;
- *created_at* — момент запуска;
- *description* — краткий текстовый комментарий или метка сценария.

Здесь создаются индексы по *user*, *created_at* и по часто используемым параметрам (например, по значению параметра связи или диапазону ρ), поскольку именно по ним выполняется основная фильтрация списка запусков на странице истории экспериментов. Для поиска конкретного запуска по идентификатору достаточно встроенного индекса по *_id*.

Коллекция *lorenz_reports* хранит отчёты и агрегированные статистики по запускам. Документ отчёта содержит:

- *run_id* — ссылку на документ в *lorenz_runs*;

– *summary* – вложенный объект со сводными метриками (значения показателей Ляпунова, средние и максимальные отклонения между траекториями, показатели синхронизации и др.);

– *files* – массив объектов с описанием выводимых файлов (*type*, *path*, возможно – *format* и *size*);

– *created_at* – дата формирования отчёта;

– *author* – ссылка на пользователя, создавшего отчёт.

Для ускорения выборки отчётов по запуску создаётся индекс по полю *run_id*; индекс по *created_at* используется для отображения последних отчётов.

Коллекция *publications* отвечает за хранение библиографических описаний статей и отчётов. Документ включает:

– *title* – название работы;

– *authors* – строка или массив авторов;

– *type* – тип публикации (статья журнала, тезисы конференции и т.п.);

– *journal*, *year*, *doi* и другие реквизиты;

– *keywords*, *abstract*;

– *created_at*, *updated_at*.

По полям *title* и *year* может быть создан составной индекс для ускорения поиска и сортировки, а по *keywords* – текстовый индекс, если планируется полнотекстовый поиск.

Коллекция *literature* содержит ссылки на использованные источники (книги, статьи, веб-ресурсы). Документы включают *URL*, библиографическое описание по ГОСТ или иному стандарту, а также временные метки создания и изменения. Индекс по *url* обеспечивает уникальность и быстрый поиск ссылок.

Коллекция *figures* хранит сведения о сохранённых изображениях и других графических материалах, полученных при моделировании. Типичный документ содержит путь к файлу, тип изображения (фазовый портрет, временной ряд, спектр и т.д.), ссылки на соответствующий запуск (*run_id*) и отчёт (*report_id*), а также метаданные (разрешение, формат, дата создания). Индексы по *run_id* и *report_id* ускоряют нахождение всех рисунков, связанных с конкретным экспериментом или отчётом.

Коллекция *logs* используется для записи технических событий и действий пользователей: входы в систему, неудачные попытки аутентификации, запуск и завершение моделирований, ошибки и исключения. Документы содержат тип события, пользователя (если применимо), временную метку и текстовое описание. Индекс по *created_at* и *event_type* позволяет эффективно просматривать журнал и фильтровать записи по типу события или интервалу времени.

Создание индексов в *MongoDB* является аналогом проектирования ключей и индексов в реляционной базе и напрямую влияет на скорость выполнения запросов. Для каждой коллекции выделяются поля, по которым чаще всего выполняются фильтрация и сортировка, и по ним создаются одиночные или составные индексы.

- *users* – уникальный индекс по *username* и дополнительный индекс по *role*, что ускоряет аутентификацию и административные операции.

- *lorenz_runs* – индексы по *user*, *created_at*, ключевым параметрам моделирования и, при необходимости, по полю статуса запуска. Это улучшает работу страницы истории запусков и выборку данных для отчётности.

- *lorenz_reports* – индексы по *run_id* и *created_at* для быстрого получения отчётов по эксперименту и вывода последних отчётов в интерфейсе.

- *publications* и *literature* – индексы по названию, году и *URL*, а при необходимости – текстовый индекс для поиска по ключевым словам.

- *figures* – индексы по *run_id* и *report_id*, позволяющие быстро находить все изображения для выбранного запуска или отчёта.

- *logs* – индекс по *created_at* и *event_type* для удобства анализа логов и построения выборок по типу событий.

Использование индексов особенно важно, поскольку в коллекции запусков и отчётов потенциально может накапливаться большое количество документов, а пользователи активно фильтруют и сортируют данные в веб-интерфейсе.

В документно-ориентированной модели жёсткие внешние ключи отсутствуют, а логическая целостность поддерживается на уровне прикладного кода. При удалении пользователя не допускается удаление связанных экспериментов и отчётов: используется логическое удаление (статус) или передача экспериментов другому пользователю. Для связок «запуск–отчёт», «запуск–рисунок», «публикация–запуск» применяются идентификаторы и проверки существования ссылок при критических операциях.

Вместо отдельных архивных таблиц используется флаг статуса (*active*, *archived*) и индекс по этому полю, что позволяет отделять актуальные записи от исторических без дублирования коллекций. При необходимости старые данные могут быть перенесены в архивные коллекции, но на уровне курсового проекта достаточно логической маркировки и фильтрации в интерфейсе.

Часть работы с коллекциями реализована во вспомогательных функциях на *Python*, играющих роль аналогов триггеров и событий: при запуске создаётся документ в *lorenz_runs*, по завершении расчёта – запись в

lorenz_reports, при добавлении публикации или источника литературы – документы в *publications* и *literature* с обновлением поля *updated_at*.

Систематическое использование вспомогательных функций позволяет централизованно контролировать формат документов и поддерживать единообразие данных во всех коллекциях. Все изменения проходят через слой бизнес-логики на *Python*, где выполняются дополнительные проверки, валидация и логирование.

В результате физическая реализация базы данных на *MongoDB* соответствует логической модели, учитывает особенности предметной области и поддерживает эффективную работу приложения: быстрый доступ к параметрам и результатам запусков, удобную навигацию по отчётам и публикациям, а также надёжное хранение и анализ технических журналов работы системы [12].

Спроектированная физическая схема базы данных для СУБД представлена на рисунке 3.1.

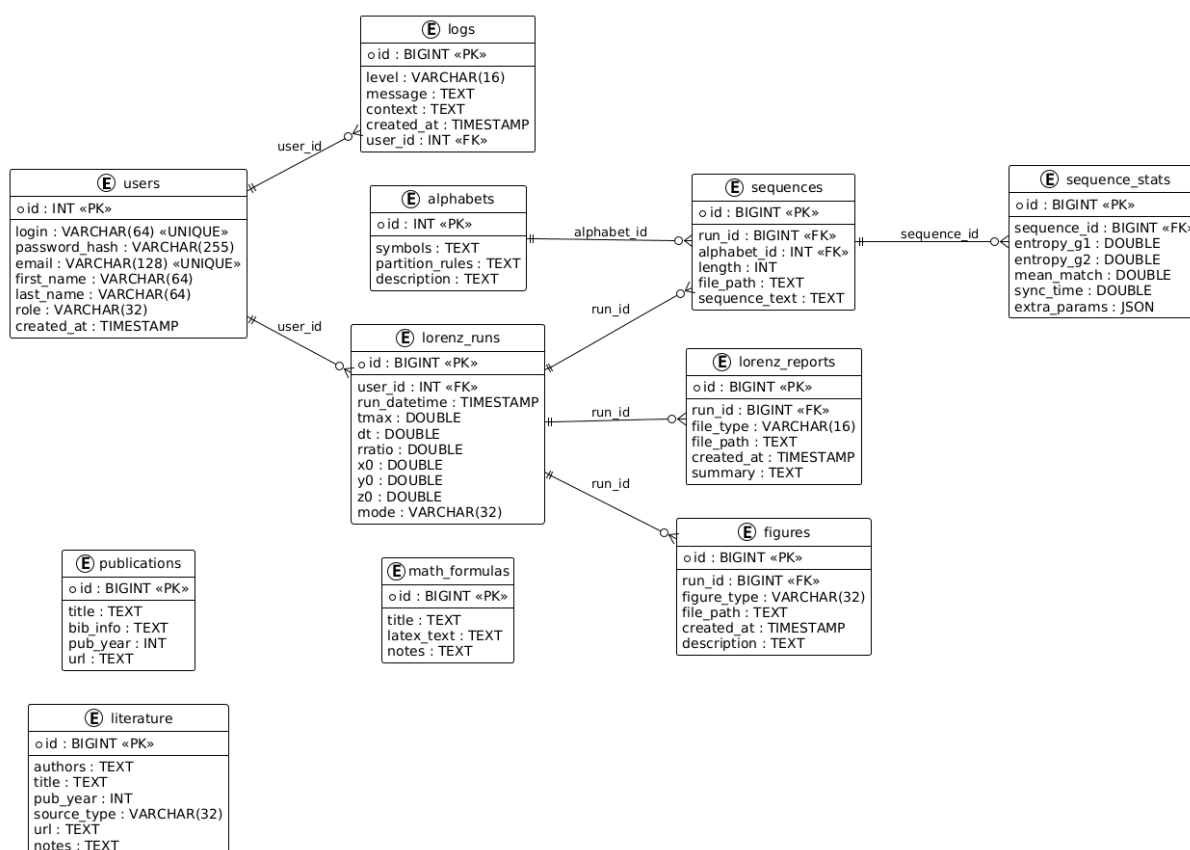


Рисунок 3.1 – Физическая схема базы данных

Сущность *User* хранит информацию обо всех пользователях системы анализа хаотических режимов Лоренца. Поля сущности и их назначение приведены в таблице 3.1.

Таблица 3.1 – Описание таблицы *User*.

Название поля	Тип данных	Описание	Примечание
<i>id</i>	<i>INT</i>	Идентификатор пользователя	Первичный ключ
<i>login</i>	<i>VARCHAR(64)</i>	Логин пользователя	Уникальный
<i>password_hash</i>	<i>VARCHAR(255)</i>	Хеш-сумма пароля пользователя	
<i>first_name</i>	<i>VARCHAR(64)</i>	Имя пользователя	
<i>last_name</i>	<i>VARCHAR(64)</i>	Фамилия пользователя	
<i>email</i>	<i>VARCHAR(128)</i>	Адрес электронной почты пользователя	Уникальный
<i>role</i>	<i>VARCHAR(32)</i>	Роль пользователя (пользователь, администратор)	
<i>email_confirmed</i>	<i>BOOL</i>	Признак подтверждения адреса электронной почты	
<i>confirmation_code</i>	<i>VARCHAR(16)</i>	Код подтверждения, отправляемый для активации	
<i>created_at</i>	<i>DATETIME</i>	Дата и время создания учётной записи	

Сущность *lorenz_runs* хранит данные о параметрических запусках модели Лоренца: завершённых, выполнявшихся ранее и запланированных. Описание сущности приведено в таблице 3.2.

Таблица 3.2 – Описание таблицы *Lorenz_runs*.

Название поля	Тип данных	Описание	Примечание
<i>id</i>	<i>BIGINT</i>	Идентификатор запуска	Первичный ключ
<i>user_id</i>	<i>INT</i>	Идентификатор пользователя, инициировавшего запуск	Внешний ключ
<i>run_datetime</i>	<i>DATETIME</i>	Дата и время запуска моделирования	
<i>tmax</i>	<i>DOUBLE</i>	Время моделирования <i>tmax</i>	
<i>dt</i>	<i>DOUBLE</i>	Шаг интегрирования <i>dt</i>	
<i>rratio</i>	<i>DOUBLE</i>	Параметр <i>r</i>	
<i>x0</i>	<i>DOUBLE</i>	Начальное значение координаты <i>x</i>	<i>x0</i>

Продолжение таблицы 3.2

Название поля	Тип данных	Описание	Примечание
<i>y0</i>	<i>DOUBLE</i>	Начальное значение координаты <i>y</i>	
<i>z0</i>	<i>DOUBLE</i>	Начальное значение координаты <i>z</i>	
<i>mode</i>	<i>VARCHAR(32)</i>	Режим моделирования	

Сущность *Alphabets* хранит описание используемых символьных алфавитов. Её структура приведена в таблице 3.3.

Таблица 3.3 – Описание таблицы *Alphabets*.

Название поля	Тип данных	Описание	Примечание
<i>id</i>	<i>INT</i>	Идентификатор алфавита	Первичный ключ
<i>symbols</i>	<i>TEXT</i>	Набор символов	
<i>partition_rules</i>	<i>TEXT</i>	Правила разбиения фазового пространства	
<i>description</i>	<i>TEXT</i>	Текстовое описание	

Sequences хранит символьные последовательности, полученные по результатам запусков. Структура сущности представлена в таблице 3.4.

Таблица 3.4 – Описание таблицы *Sequences*.

Название поля	Тип данных	Описание	Примечание
<i>id</i>	<i>BIGINT</i>	Идентификатор последовательности	Первичный ключ
<i>run_id</i>	<i>BIGINT</i>	Идентификатор запуска, из которого получена последовательность	Внешний ключ
<i>alphabet_id</i>	<i>INT</i>	Идентификатор алфавита, по которому выполнено кодирование	Внешний ключ
<i>length</i>	<i>INT</i>	Длина последовательности	
<i>file_path</i>	<i>TEXT</i>	Путь к файлу с сохранённой последовательностью	
<i>sequence_text</i>	<i>TEXT</i>	Символьное представление последовательности	

Сущность *sequence_stats* хранит статистические характеристики, вычисленные по последовательностям. Её описание приведено в таблице 3.5.

Таблица 3.5 – Описание таблицы *Sequence_stats*.

Название поля	Тип данных	Описание	Примечание
<i>id</i>	<i>BIGINT</i>	Идентификатор записи статистики	Первичный ключ
<i>sequence_id</i>	<i>BIGINT</i>	Идентификатор последовательности	Внешний ключ
<i>entropy_g1</i>	<i>DOUBLE</i>	Энтропия G1	
<i>entropy_g2</i>	<i>DOUBLE</i>	Энтропия G2	
<i>mean_match</i>	<i>DOUBLE</i>	Средняя доля совпадений последовательностей	
<i>sync_time</i>	<i>DOUBLE</i>	Время синхронизации	
<i>extra_params</i>	<i>JSON</i>	Дополнительные параметры (распределения и др.)	

Сущность *lorenz_reports* хранит сведения о сформированных отчётах по запускам. Структура сущности показана в таблице 3.6.

Таблица 3.6 – Описание таблицы *Lorenz_reports*.

Название поля	Тип данных	Описание	Примечание
<i>id</i>	<i>BIGINT</i>	Идентификатор отчёта	Первичный ключ
<i>run_id</i>	<i>BIGINT</i>	Идентификатор запуска, к которому относится отчёт	Внешний ключ
<i>file_type</i>	<i>VARCHAR(16)</i>	Формат файла отчёта (<i>CSV</i> , <i>XLSX</i> , <i>DOCX</i> , <i>PDF</i>)	
<i>file_path</i>	<i>TEXT</i>	Путь к файлу отчёта	
<i>created_at</i>	<i>DATETIME</i>	Дата и время формирования отчёта	
<i>summary</i>	<i>TEXT</i>	Краткое текстовое описание отчёта	

Сущность *figures* предназначена для хранения информации о сохранённых графиках (аттракторах, спектрах, гистограммах и т.п.). Её структура приведена в таблице 3.7.

Таблица 3.7 – Описание таблицы *Figures*.

Название поля	Тип данных	Описание	Примечание
<i>id</i>	<i>BIGINT</i>	Идентификатор фигуры	Первичный ключ
<i>run_id</i>	<i>BIGINT</i>	Идентификатор запуска, для которого построена фигура	Внешний ключ
<i>figure_type</i>	<i>VARCHAR(32)</i>	Тип графика (аттрактор, фаза, спектр и др.)	
<i>file_path</i>	<i>TEXT</i>	Путь к файлу изображения	
<i>created_at</i>	<i>DATETIME</i>	Дата и время сохранения	
<i>description</i>	<i>TEXT</i>	Текстовое описание	

Для части сущностей заданы уникальные индексы. Например, в *sequences* может быть определён уникальный индекс по полям *run_id* и *alphabet_id* для предотвращения дублирования одинаковых последовательностей; в таблице *sequence_stats* – по полю *sequence_id*, если для каждой последовательности хранится один набор основных статистик; в *users* – по полям *login* и *email*. Уникальные индексы обеспечивают целостность данных и ускоряют выполнение запросов поиска по соответствующим комбинациям столбцов.

Помимо уникальных индексов используются и обычные. Так, для сущности *lorenz_runs* создаётся индекс по полю *run_datetime*, поскольку список запусков по умолчанию выводится в хронологическом порядке; наличие индекса повышает производительность таких запросов. Индексы создаются и для полей, являющихся внешними ключами (*user_id*, *run_id*, *alphabet_id*, *sequence_id* и т.д.) для ускорения соединений таблиц и контроля ссылочной целостности при вставке и удалении записей.

Разделение данных на специализированные таблицы (*lorenz_runs*, *sequences*, *sequence_stats*, *lorenz_reports*, *figures*) позволяет эффективно работать с большими объёмами информации: запросы к актуальным результатам и отчётам не затрагивают вспомогательные или исторические данные, что положительно сказывается на производительности.

В базе данных могут использоваться триггеры и фоновые задания, автоматизирующие обслуживание данных. Например, триггер при вставке в *lorenz_runs* может инициировать предварительную инициализацию связанных записей, а периодическое событие – архивировать старые запуски или удалять временные данные. Для внешних ключей при необходимости задаётся каскадное обновление и удаление: при удалении запуска удаляются связанные

с ним последовательности, статистики, отчёты и фигуры, что предотвращает появление «висячих» ссылок.

Безопасность хранения персональных данных обеспечивается за счёт сохранения не самих паролей, а их криптографических хеш-значений в поле *password_hash*, а также за счёт контроля уникальности и подтверждения адреса электронной почты пользователя.

3.3 Программная реализация приложения

После разработки структуры базы данных и её наполнения экспериментальными и справочными данными начинается этап программной реализации веб приложения для анализа хаотических режимов Лоренца. Цель модуля заключается в создании возможности интерактивного запуска моделирования и анализа динамики системы через веб интерфейс без обращения к серверному коду. Архитектура приложения основана на фреймворке *Dash*, который объединяет визуальную часть и серверную логику обработки данных в одной среде *Python*. Реализованы три уровня логики: интерфейсный, доменный и вспомогательный. Интерфейсный уровень отвечает за работу с пользователем, доменный содержит численные методы интегрирования и взаимодействие с базой данных, а вспомогательные модули обеспечивают экспорт отчётов и ведение журнала вычислительных сессий.

Пример функции запуска моделирования Лоренца и сохранения результатов:

```
@app.callback(  
    Output("lorenz-analysis-store", "data"),  
    Output("save-msg", "children"),  
    Input("analyze-btn", "n_clicks"),  
    State("alphabet-dropdown", "value"),  
    State("param-tmax", "value"),  
    State("param-dt", "value"),  
    State("param-r", "value"),  
    State("symbol-input", "value"),  
    prevent_initial_call=True,  
)  
def run_analysis_cb(n_clicks, alphabet_choice, tmax, dt, rratio, symbol):  
    if not n_clicks:  
        raise dash.exceptions.PreventUpdate  
  
    tmax = tmax or TMAX
```

```

dt = dt or DT
rratio = rratio or RRATIO
symbol = symbol or "A0"
alphabet_choice = alphabet_choice or "ascii"

# доменная функция анализа
analysis = analyzelorenzrun(
    tmax=float(tmax),
    dt=float(dt),
    rratio=float(rratio),
    alphabetchoice=alphabet_choice,
    symbolofinterest=symbol,
    mode="app4",
)

params_run = {
    "tmax": float(tmax),
    "dt": float(dt),
    "r": float(rratio),
    "x0": 0.1,
    "y0": 0.0,
    "z0": 0.0,
}

run_id = createlorenzrun(params=params_run, user="anonymous")
csv_path, length = exportcsvfromanalysis(analysis, prefix="lorenz_summary")

summary = {
    "name": f"App4 {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
    "mode": "App4",
    "filetype": "csv",
    "alphabet": "".join(analysis["params"]["alphabet"]),
    "tmax": analysis["params"]["tmax"],
    "dt": analysis["params"]["dt"],
    "rratio": analysis["params"]["rratio"],
    "symbol": analysis["params"]["symbol"],
    "meanmatch": analysis["meanmatch"],
    "synctimes": analysis["synctimes"],
    "entropyg1": analysis["H1"],
    "entropyg2": analysis["H2"],
}

```

```

    "filepath": csv_path,
}

createlorenzreport(run_id, summary=summary, filepath=csv_path)

msg = f"CSV-отчёт создан (длина последовательности: {length})"
return analysis, msg

```

После выполнения этой функции система сохраняет результаты моделирования, включая меры энтропии, синхронизационные интервалы и основные характеристики траекторий. Сформированный CSV-файл помещается в репозиторий научных материалов и доступен пользователю для анализа или включения в отчёт. Такой подход повышает воспроизводимость экспериментов и прозрачность данных. Механизм реализован через функцию обратного вызова, реагирующую на нажатие кнопки «Анализировать». При активации выполняется моделирование системы Лоренца с указанными параметрами, результаты сохраняются в базе данных и формируется CSV-отчёт, что обеспечивает обновление интерфейса.

Функция запускает численное моделирование двух систем Лоренца с заданными параметрами и начальными условиями. После анализа результаты сохраняются в структуре *analysis*, создаётся запись о запуске в таблице *lorenz_runs*, а CSV-отчёт и сведения о нём добавляются в таблицу *lorenz_reports*. Пользователь получает уведомление об успешном завершении анализа, а данные временно кэшируются в хранилище *Dash* для дальнейшего экспорта и последующей обработки.

Пример функции экспорта результатов анализа:

```

@app.callback(
    Output("save-msg", "children"),
    Input("export-excel-btn", "n_clicks"),
    Input("export-word-btn", "n_clicks"),
    Input("export-pdf-btn", "n_clicks"),
    State("lorenz-analysis-store", "data"),
    State("tabs", "value"),
    prevent_initial_call=True,
)
def manual_export_cb(n_excel, n_word, n_pdf, analysis, current_tab):
    if analysis is None:
        raise dash.exceptions.PreventUpdate

```

```

ctx = callbackcontext
if not ctx.triggered:
    raise dash.exceptions.PreventUpdate

trigger = ctx.triggered[0]["prop_id"].split(".")[0]
mode_desc = "App3" if current_tab == "app3" else "App4"

if trigger == "export-excel-btn":
    path = exportexcelfromanalysis(analysis, prefix="lorenz")
    return f"Excel-отчёт сохранён: {path}"

if trigger == "export-word-btn":
    path = exportwordfromanalysis(analysis, modedesc=mode_desc, prefix="lorenz")
    return f"Word-отчёт сохранён: {path}"

if trigger == "export-pdf-btn":
    path = exportpdffromanalysis(analysis, prefix="lorenz")
    return f"PDF-отчёт сохранён: {path}"

raise dash.exceptions.PreventUpdate

```

Функция экспорта вручную использует уже выполненный анализ, сохранённый в памяти, и по нажатию соответствующей кнопки создаёт отчёт в формате *Excel*, *Word* или *PDF*, используя специализированные функции модуля экспорта. Путь к сформированному файлу возвращается в виде текстового сообщения пользователю, что позволяет быстро понять, где сохранён отчёт и был ли экспорт успешным:

```

@app.route("/download_run_pdf/<run_id>")
@login_required
def download_run_pdf(run_id: str):
    # Загрузка данных о запуске и отчёте из Mongo/БД
    run = get_lorenz_run_by_id(run_id)      # доменная функция
    report = get_lorenz_report_by_run(run_id) # метаданные отчёта (если нужно)

    if not run:
        flash("Запуск не найден.", "danger")
        return redirect(url_for("lorenz_runs_page"))

    # Выполняем анализ или берём сохранённый

```

```

analysis = rebuild_or_load_analysis(run) # по желанию

pdf_path = exportpdffromanalysis(analysis, prefix="lorenz") # domain.lorenz.exports

return send_file(
    pdf_path,
    as_attachment=True,
    download_name=f"lorenz_run_{run_id}.pdf",
    mimetype="application/pdf",
)

```

Перед генерацией отчёта функция проверяет существование запрошенного запуска и права доступа текущего пользователя. При отсутствии данных или нарушении прав возвращается сообщение об ошибке и выполняется перенаправление на страницу со списком запусков. При успешной проверке отчёт создаётся (или переиспользуется уже существующий файл) и становится доступен для повторной загрузки без повторного анализа.

Администратор системы имеет доступ к аналитике по выполненным запускам и сформированным отчётам. Он может запросить, например, график количества запусков за период или динамику средней энтропии:

```

def generate_runs_chart(start_date: date, end_date: date):
    try:
        # Пример для MongoDB: агрегирование запусков по дате
        pipeline = [
            {"$match": {
                "run_datetime": {
                    "$gte": datetime.combine(start_date, time.min),
                    "$lte": datetime.combine(end_date, time.max),
                }
            }},
            {"$group": {
                "_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$run_datetime"}},
                "count": {"$sum": 1},
            }},
            {"$sort": {"_id": 1}},
        ]
        data = list(mongo_runs_collection.aggregate(pipeline))
    
```

```

if not data:
    flash("Нет данных для построения графика.", "warning")
    return redirect(url_for("reports_page"))

dates = [row["_id"] for row in data]
counts = [row["count"] for row in data]

import io, base64
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 4))
plt.plot(dates, counts, marker="o", color="b", label="Запуски")
plt.title("Количество запусков Лоренца за период")
plt.xlabel("Дата")
plt.ylabel("Число запусков")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()

img_stream = io.BytesIO()
plt.savefig(img_stream, format="png")
img_stream.seek(0)

graph_url = base64.b64encode(img_stream.getvalue()).decode("utf-8")
return render_template("reports.html",
                       graph_url=f"data:image/png;base64,{graph_url}")
except Exception as e:
    flash(f"Ошибка при построении графика: {e}", "danger")
    return redirect(url_for("reports_page"))

```

Функция выбирает и агрегирует данные по запускам за указанный администратором период, группируя их по дате. При отсутствии данных выполнение дальнейших вычислений прекращается, и администратор получает соответствующее уведомление. При наличии данных формируется график, который сохраняется в поток байтов, кодируется в base64 и отображается на веб-странице отчётов. Предусмотрена обработка исключений, позволяющая корректно отреагировать на ошибки запросов и визуализации.

4 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Начальная страница входа в веб-приложения имеет вид, представленный на рисунке 4.1.

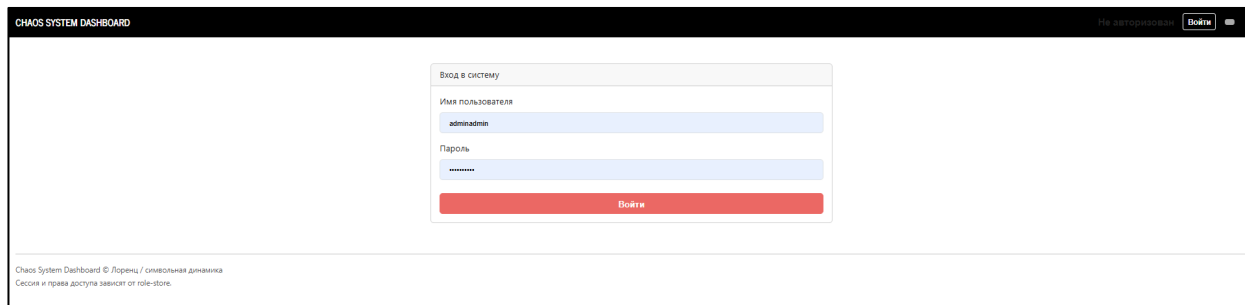


Рисунок 4.1 – Начальная страница веб-приложения

Как видно, неавторизованный пользователь должен войти в приложение введя логин и пароль. Неавторизованному пользователю никакой функционал недоступен.

После авторизации пользователь получит доступ к элементам панели навигации и функциям системы, в полном объеме соответствующим функциям пользователя, описанным в диаграмме вариантов использования.

Внешний вид страницы для авторизованного пользователя (с правами администратора) представлен на рисунке 4.2.

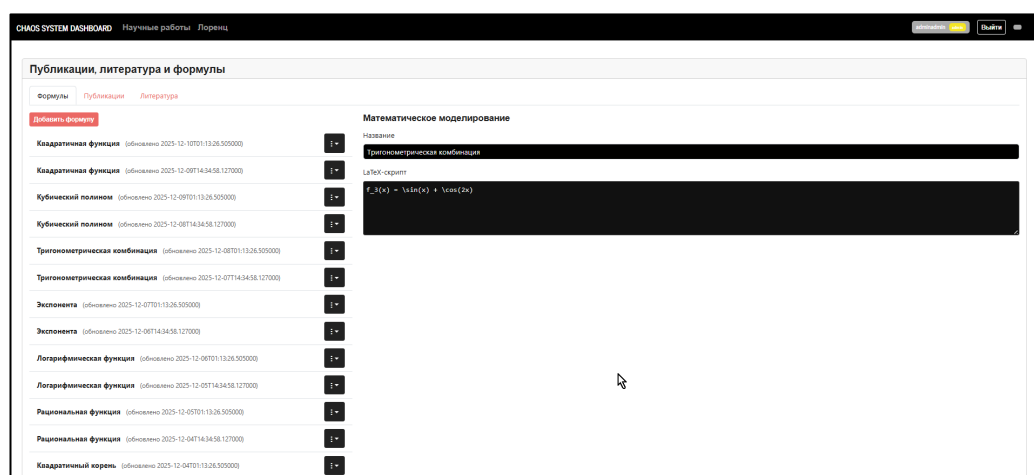


Рисунок 4.2 – Внешний вид страницы для авторизованного пользователя

В системе анализа хаотических режимов Лоренца одной из основных вкладок является вкладка «Лоренц». На этой странице пользователю отображается форма для задания параметров моделирования (времени

интегрирования, шага, параметра r , начальных условий), а также выбора режима приложения и символического алфавита. После ввода параметров пользователь может запустить моделирование и увидеть результаты в виде графиков аттрактора, временных рядов и краткого текстового резюме. Внешний вид вкладки «Лоренц» показан на рисунке 4.3.

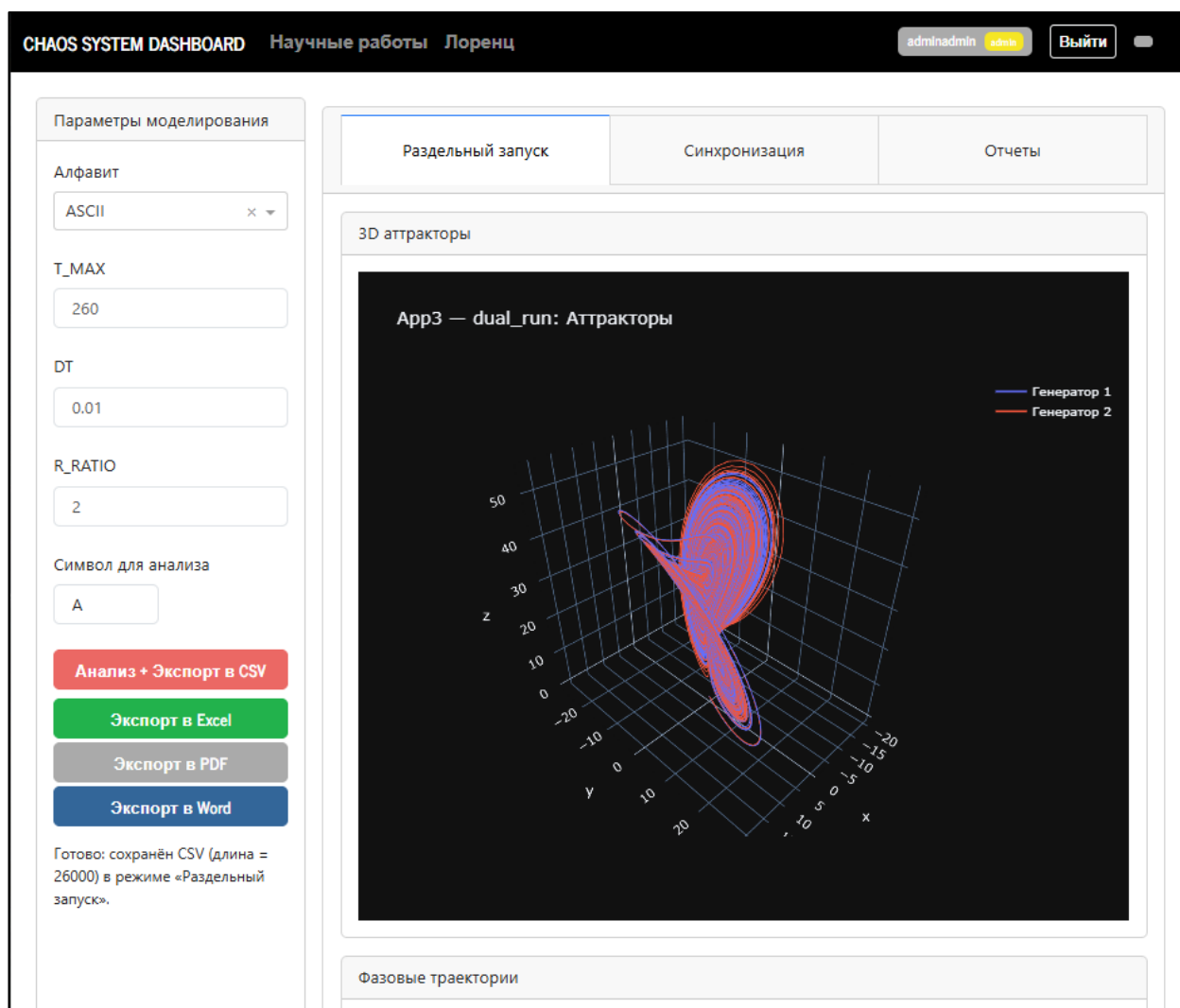


Рисунок 4.3 – Вкладка «Лоренц»

Второй не менее важной вкладкой является вкладка «Отчеты». На данной странице отображается список всех выполненных пользователем запусков системы Лоренца с указанием даты и времени, параметров моделирования и режима работы. Запуски могут быть отсортированы и отфильтрованы, например, по дате или по режиму. Внешний вид вкладки «Отчеты» представлен на рисунке 4.4.

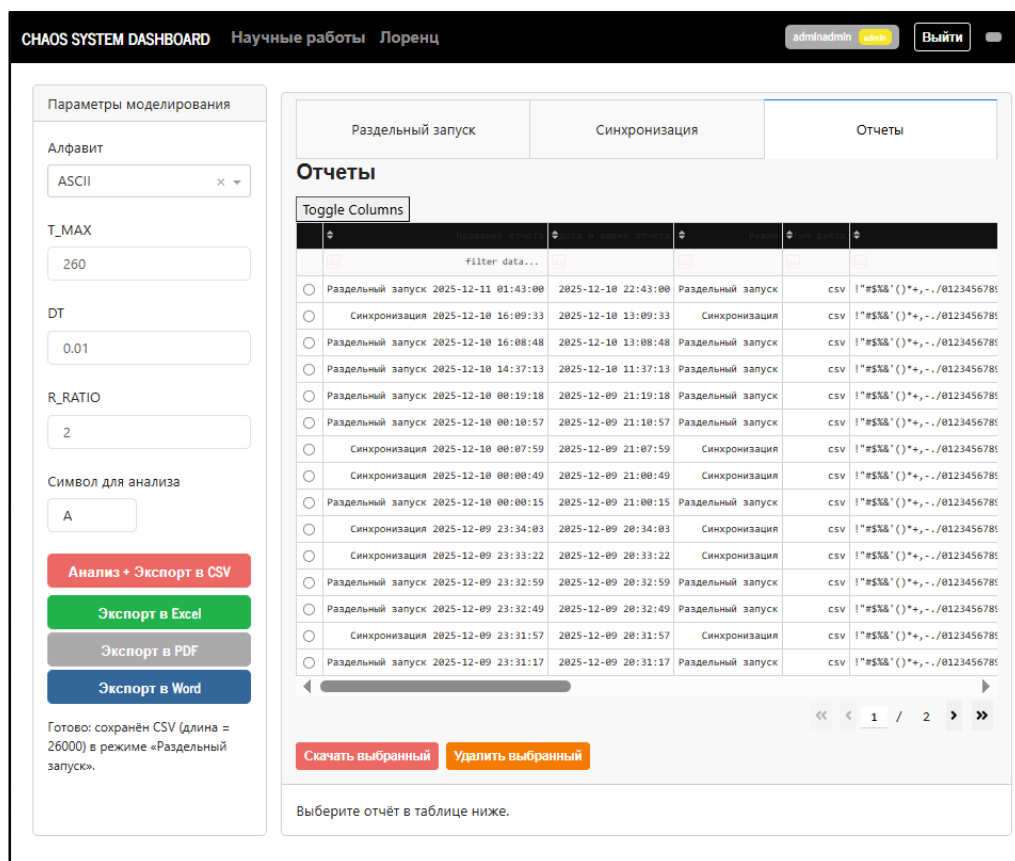


Рисунок 4.4 – Вкладка «Отчеты»

На странице детального просмотра запуска пользователь может сформировать и скачать отчёты в различных форматах. Соответствующие кнопки инициируют вызов функций модуля экспорта, которые создают файлы *CSV*, *Excel*, *Word* или *PDF* на основе уже вычисленного анализа. Путь к сформированному файлу отображается в виде сообщения, а при экспорте *PDF* отчёт также доступен напрямую для скачивания через *HTTP*-ответ. Пример блока со списком отчётов и кнопками экспорта приведён на рисунке 4.5.

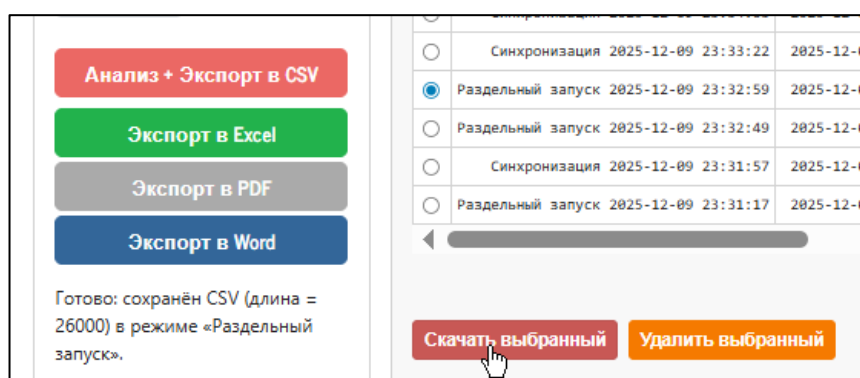


Рисунок 4.5 – Экспорт отчётов по запуску

Отдельная вкладка «Публикации» предоставляет пользователю доступ к базе публикаций и литературных источников, связанных с исследуемыми

режимами хаоса. Записи выводятся в виде таблицы с основными полями (авторы, название, год, ссылка), предусмотрен поиск по тексту и фильтрация по типу источника. Пользователь может открыть карточку записи и при необходимости перейти по внешней ссылке на оригинал работы. Внешний вид этой вкладки показан на рисунке 4.6.

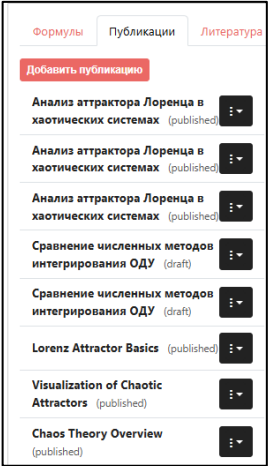


Рисунок 4.6 – Список публикаций

Во вкладке «Формулы» отображаются математические выражения, используемые в описании метода анализа, в виде списка с заголовками и рендерингом *LaTeX*. Пользователь может просмотреть текст формулы, при необходимости выбрать её для вставки в отчёт или сопоставить с используемыми параметрами моделирования. Эта вкладка иллюстрируется на рисунке 4.7.

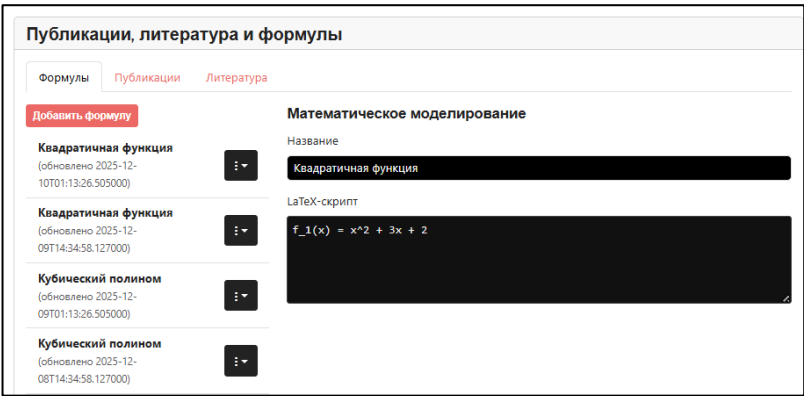


Рисунок 4.7 – Вкладка с формулами

Пользователю доступен функционал выхода из системы, смены темы (светлая/темная) и просмотр своего имени пользователя и роли (представлен на рисунке 4.8).



Рисунок 4.8 – Панель навигации

Также администратору доступно добавление новых сущностей и удаление старых. На рисунке 4.9 представлен функционал добавления новой формулы.

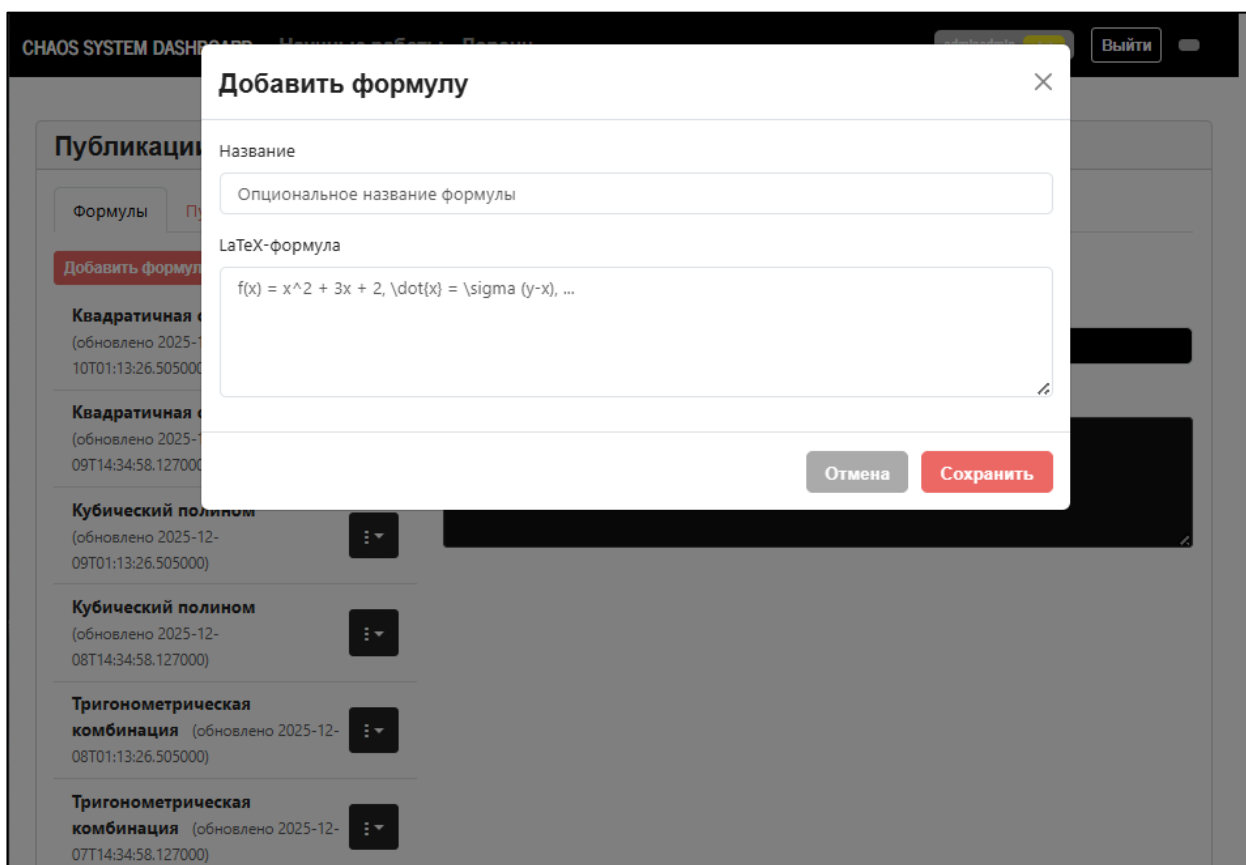


Рисунок 4.9 – Окно добавления формулы

При корректном заполнении всех обязательных полей и успешной проверке данных после нажатия кнопки сохранения система возвращает пользователя на форму и выводит в верхней части страницы информативное сообщение о результате операции (см. рисунок 4.11). В уведомлении указывается, что запись была успешно добавлена или обновлена, что подтверждает сохранение введенных параметров в базе данных и позволяет пользователю сразу убедиться в отсутствии ошибок. Такое поведение интерфейса повышает наглядность работы системы и снижает вероятность повторного ввода одних и тех же данных.

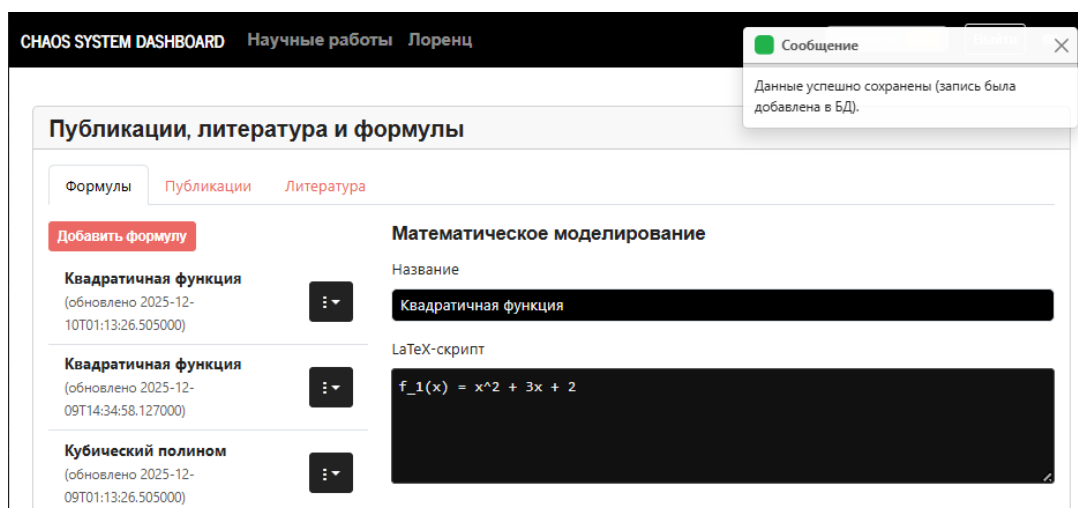


Рисунок 4.10 – Уведомление успешного добавления

По нажатию на «три точки» открывается выпадающее меню с функциями удаления и редактирования (см. рисунок 4.11).

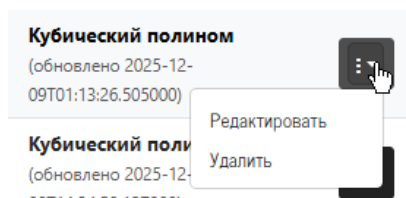


Рисунок 4.11 – Открытие выпадающего меню

Если пользователь выбрал удаление, то система покажет пользователю окно подтверждения удаления сущности (см. рисунок 4.12)

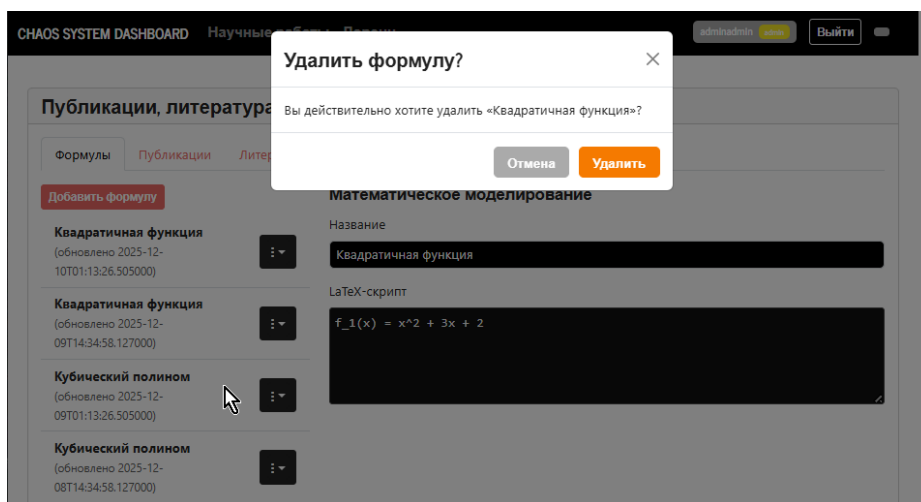


Рисунок 4.12 – Открытие выпадающего меню

После подтверждения система удалит выбранную сущность отправив пользователю уведомление об удалении. Если пользователь выбрал удаление,

то система покажет пользователю окно подтверждения удаления сущности (см. рисунок 4.13)

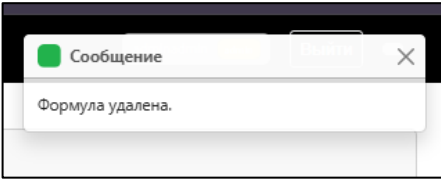


Рисунок 4.13 – Уведомление об удалении

Скачать отчеты запусков можно по нажатию на кнопку «Скачать выбранный», тогда система скачает выбранный отчет и пользователь увидит его в загрузках (см. рисунок 4.14)

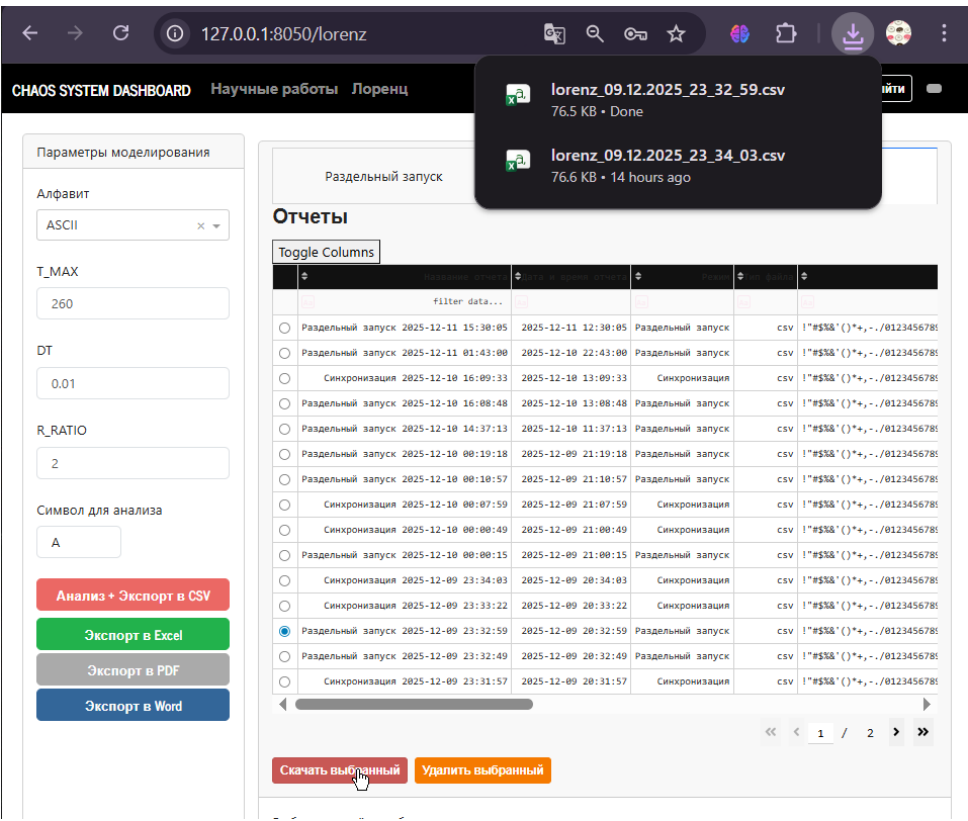


Рисунок 4.16 – Сохранение отчета

После завершения загрузки пользователь может открыть полученный файл отчёта локальными средствами операционной системы и просмотреть рассчитанные параметры, графики и текстовое описание эксперимента. При необходимости тот же отчёт может быть повторно выгружен в дальнейшем, поскольку информация о сформированных файлах и их путях сохраняется в базе данных и отображается в интерфейсе приложения.

ЗАКЛЮЧЕНИЕ

В рамках курсового проекта по разработке информационной системы для исследования генератора Лоренца был выполнен комплексный анализ предметной области, включающий изучение хаотических динамических систем, обзор существующих веб-симуляций и визуализаторов аттрактора Лоренца, а также анализ их сильных и слабых сторон. На основе этого анализа были сформулированы задачи проекта и требования к разрабатываемому приложению: поддержка запусков численного моделирования, хранение параметров и результатов экспериментов, ведение библиотеки публикаций и литературы, формирование отчётов и организация ролевого доступа пользователей к данным.

В процессе работы была спроектирована концептуальная и логическая модель данных, отражающая основные сущности системы (пользователи, запуски генератора, отчёты, публикации, источники литературы, сохранённые изображения, журналы событий) и связи между ними, после чего она была преобразована в физическую структуру базы данных на основе документно-ориентированной СУБД *MongoDB*, где каждая сущность реализована в виде отдельной коллекции с продуманной системой индексов и полей-ссылок, что позволило учесть особенности исследовательских данных, включая вложенные структуры и эволюционирующую схему документов.

Для серверной части выбран язык *Python* с использованием стека *Flask/Dash*, что обеспечило возможность прямого переиспользования численных алгоритмов интегрирования системы Лоренца, расчёта показателей Ляпунова и анализа синхронизации, а также упростило построение интерактивного веб-интерфейса; клиентская часть реализована на основе компонентов *Dash*, *HTML* и *CSS*, обеспечивая удобную работу с параметрами моделирования, историей запусков, визуализацией результатов и отчётами при наличии механизмов аутентификации, авторизации и журналирования действий.

Разработанное веб-приложение предоставляет пользователю инструменты для запуска и анализа численных экспериментов, формирования отчётов и их связи с научными публикациями, а администратору – средства управления учётными записями и каталогами данных, что позволяет считать цели и задачи курсового проекта достигнутыми. Система соответствует основным требованиям к выпускным работам: выполнен анализ предметной области, обоснован выбор архитектуры и технологий, спроектированы и реализованы база данных и прикладной уровень, подготовлена пользовательская документация, намечены направления дальнейшего развития и использования в учебной и исследовательской практике [13].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Lorenz, E. N. *Deterministic Nonperiodic Flow* // *Journal of the Atmospheric Sciences*. – 1963. – Vol. 20, No. 2. – P. 130–141.
- [2] *Three.js Lorenz Attractor* [Электронный ресурс]. – Режим доступа : <https://threejs.org/>.
- [3] *Wolfram Demonstrations Project: Lorenz Attractor* [Электронный ресурс]. – Режим доступа : <https://demonstrations.wolfram.com/>.
- [4] Осипов, Д. Технологии проектирования баз данных. / Д. Осипов. – М. : ДМК Пресс, 2019. – 498 с.
- [5] Фаулер, М. *UML. Основы: краткое руководство по стандартному языку объектного моделирования*. / М. Фаулер. – спб. : Символ-Плюс, 2013. – 192 с.
- [6] Леоненков А. В. Самоучитель *UML 2* : Леоненков А. В. – спб.: БХВ-Петербург, 2007. – 576 с.
- [7] Диги, С. М. Базы данных: проектирование и использование: учебник / С. М. Диги. – М. : Финансы и статистика, 2005.— 592 с.
- [8] Хавербеке, М. Выразительный *javascript*. Современное веб-программирование. / Марейн Хавербеке. – спб. : ООО "Питер Мейл", 2021. – 480 с.
- [9] Дакет, Дж. *HTML и CSS. Разработка и дизайн веб-сайтов*. / Джон Дакет. – М. : ООО «Издательство «Эксмо», 2014. – 480 с.
- [10] Грюнберг, М. Разработка веб-приложений с использованием *Flask* на языке *Python*. / пер. С англ. А. Н. Киселёва. – М. : ДМК Пресс, 2016. – 272 с.
- [11] Бирн Д. Безопасность веб-приложений на *Python* / пер. С англ. С. С. Скобелева, А. Н. Киселева. – М. : ДМК Пресс, 2023. – 334
- [12] Стружкин Н. П. Базы Данных: проектирование: учебник для академического бакалавриата / Н. П. Стружкин, В. В. Годин. – М. : Издательство Юрайт, 2017. – 477 с.
- [13] СТП 01-2024 Стандарт предприятия. Дипломные проекты (работы). Общие требования. – Минск : БГУИР, 2024 – 178 с.

ПРИЛОЖЕНИЕ А
(обязательное)
Программный код функции приложения

```
from __future__ import annotations

from typing import Optional, Dict, Any, TypedDict

import dash
from dash import html, dcc, Input, Output, State
import dash_bootstrap_components as dbc

from storage import init_indexes
from layouts.common import build_navbar, build_footer
from auth import build_login_layout, register_auth_callbacks
from pages.papers.callbacks_core import register_papers_callbacks
from pages.lorenz.callbacks import register_lorenz_callbacks
from pages.lorenz.reports_callbacks import register_reports_callbacks
from pages.papers.layout import build_papers_layout
from pages.lorenz.layout import build_lorenz_layout


# =====
# Тупы
# =====

class RoleData(TypedDict, total=False):
    username: str
    role: str
    isadmin: bool
    is_admin: bool


# =====
# Константы приложения
# =====

BOOTSWATCH_BASE = "https://cdn.jsdelivr.net/npm/bootswatch@5.3.0/dist"
```

```
return THEME_URLS.get(name, THEME_URLS[DEFAULT_THEME])
```

```
@app.callback(
    Output("theme-toggle-btn", "children"),
    Input("theme-store", "data"),
    prevent_initial_call=False,
)
def update_theme_icon(theme_name: Optional[str]):
    name = (theme_name or DEFAULT_THEME).upper()
    # JOURNAL = светлая тема -> кнопка показывает луну (переключиться в
    # тёмную)
    return "🌙" if name == "JOURNAL" else "☀️"
```

```
# =====
# Роутинг
# =====
```

```
@app.callback(
    Output("page-content", "children"),
    Input("url", "pathname"),
    State("role-store", "data"),
    prevent_initial_call=False,
)
def display_page(pathname: Optional[str], role_data: Optional[RoleData]):
    path = pathname or LOGIN_PATH
    return _build_page_layout(path, role_data)
```

```
# =====
# Права для papers
# =====
```

```
@app.callback(
    Output("papers-is-admin", "data"),
    Input("role-store", "data"),
    prevent_initial_call=False,
```

```

)
def set_papers_admin(role_data: Optional[RoleData]):
    """
    Флаг администратора для раздела «Научные работы».
    Берётся из роли пользователя (role == 'admin' / isadmin / is_admin).
    """
    return _is_admin(role_data)


# =====
# Регистрация колбэков
# =====


register_auth_callbacks(app)
register_papers_callbacks(app)
register_lorenz_callbacks(app)
register_reports_callbacks(app)


# =====
# Точка входа
# =====


def main() -> None:
    """
    Точка входа приложения.
    Инициализирует индексы Mongo и запускает Dash-сервер.
    """
    init_indexes()
    app.run(debug=True)


if __name__ == "__main__":
    main()

```